Google Cloud Vertex AI

Vertex AI + LangChain

GCP 기반 생성형 AI 애플리케이션 구현

2026

목 차

1. 생성형 AI 이해

1.1 생성형 AI(Generative AI) 정의

1.2 생성형 AI 주요 모델

1.3 Google Cloud 의 생성형 AI 전략과 주요 서비스

2. Google Cloud Vertex AI 시작

2.1 Vertex AI 소개

2.2 Vertex AI 사용하기

2.3 Gemini 모델 상세

2.4 Gemini 멀티모달 활용

2.5 Safety Settings (안전 설정)

3. 프롬프트 엔지니어링 (Prompt Engineering)

3.1 프롬프트 엔지니어링의 개념과 중요성

3.2 효과적인 프롬프트 작성 방법

3.3 Grounding 개념

3.4 Google Search Grounding

3.5 Vertex AI Search Grounding (기업 데이터)

4. Vertex AI 로 RAG 구현

4.1 RAG 개념

4.2 벡터 데이터베이스

4.3 LangChain 으로 RAG 구현 (로컬)

4.4 Vertex AI RAG Engine

4.5 RAG 구현 방식 3 종 비교

4.6 RAG 품질 향상 기법

5. Cloud Functions + GCS 활용

5.1 Cloud Functions 으로 Vertex AI 백엔드 API 구축

5.2 GCS 기반 배포

5.3 로컬에서 테스트

6. 데이터베이스 연동

6.0 GCP 데이터베이스 서비스 개요

6.1 pgvector 개념

6.2 Cloud SQL + pgvector 로 RAG 구현

6.3 하이브리드 검색: SQL 필터 + 벡터 유사도

6.4 BigQuery + Vertex AI 연동

6.5 실습 파일 가이드

7. Vertex AI Agent Builder

7.0 AI 에이전트란?

7.1 Vertex AI Agent Builder 핵심 기능

7.2 Agent Development Kit (ADK)

7.3 AWS Bedrock Agents 와의 비교

7.4 활용 사례 및 설계 가이드

7.5 요약 및 다음 단계

부록: GCP 환경 준비 상세 가이드

부록 0-1. GCP 계정 생성 및 무료 체험

부록 0-2. 프로젝트 생성 및 API 활성화

부록 0-3. IAM 및 서비스 계정

부록 0-4. Cloud SDK(gcloud CLI) 설치

부록 0-5. VS Code + Cloud Code 설정

부록 0-6. 비용 관리

부록 0-7. 트러블슈팅

1. 생성형 AI 이해

1.1 생성형 AI(Generative AI) 정의

(1) 핵심 개념

생성형 AI 는 새로운 원본 콘텐츠(Content)를 생성할 수 있는 인공지능 기술을 말합니다. 기존 AI 가 데이터를 분류(Classification)하거나 예측(Prediction)하는 데 중점을 두었다면, 생성형 AI 는 학습한 데이터를 기반으로 텍스트, 이미지, 오디오, 코드 등 다양한 형태의 결과물을 만들어냅니다.

(2) 작동 원리

- 대규모 데이터 학습: 인터넷의 방대한 텍스트, 이미지 등의 데이터를 학습합니다.
- 패턴 인식: 데이터에 내재된 패턴, 스타일, 구조를 파악합니다.
- 생성 (Generation): 사용자의 요청(프롬프트)에 맞춰 학습한 패턴을 조합하여 새로운 결과물을 생성합니다.
- 핵심 모델: 이 모든 과정의 기반이 되는 모델을 파운데이션 모델(Foundation Model) 또는 거대 언어 모델(LLM)이라고 부릅니다.

(3) 주요 특징

- 다재다능함 (Versatility): 질문 답변, 요약, 번역, 창작, 코딩, 이미지 생성 등 활용 범위가 매우 넓습니다.
- 창의성 (Creativity): 기존에 없던 새로운 아이디어나 디자인을 생성할 수 있습니다.
- 상호작용 (Interaction): 단순 명령이 아닌, 대화형(Conversational) 소통이 가능합니다.

1.2 생성형 AI 주요 모델

현재 생성형 AI 시장은 빅테크 기업의 '파운데이션 모델' 경쟁과 특정 목적에 맞춰 효율성을 높인 '오픈소스 모델' 경쟁으로 나뉩니다.

개발사	모델명	주요 특징
Google	Gemini 2.0 / 2.5	멀티모달 성능 최상위, 긴 컨텍스트 창, 코드/분석 등 범용
Anthropic	Claude 3.5 / 4	복잡한 추론, 긴 문서 요약, 창의적 글쓰기에 탁월
Meta	Llama 3 / 4	가장 강력한 오픈소스 모델, 빠른 응답 속도
Mistral AI	Mistral Large / Small	MoE 기술로 비용 대비 성능 우수
OpenAI	GPT-4o	범용 멀티모달, 경쟁적 가격

1.3 Google Cloud 의 생성형 AI 전략과 주요 서비스

Google Cloud 는 자체 최강 모델(Gemini)을 중심에 두면서도, 기업이 다양한 모델을 선택하고 자사 데이터와 결합할 수 있는 통합 AI 플랫폼을 제공합니다.

핵심 전략: 3 계층 스택 (Three-Layer Stack)

계층	목표	핵심 서비스
1. 인프라	AI 모델 학습/추론을 위한 최적 인프라	TPU v5e/v6, NVIDIA GPU (A3/H100), Cloud GPU
2. 도구 및 플랫폼	모델 선택, 커스터마이징, 안전한 사용	Vertex AI (Model Garden, Agent Builder, RAG)
3. 애플리케이션	GCP 가 직접 만든 완성형 AI 서비스	Gemini in Workspace, Duet AI, NotebookLM

AWS Bedrock 과의 비교

구분	AWS Bedrock	GCP Vertex AI
콘셉트	파운데이션 모델 슈퍼마켓	AI 플랫폼 + Model Garden
자체 모델	Amazon Titan	Gemini
서드파티 모델	Claude, Llama, Mistral 등	Claude, Llama, Mistral 등
RAG	Knowledge Bases	Vertex AI Search / RAG API
에이전트	Bedrock Agents	Vertex AI Agent Builder
서버리스 배포	Lambda + API Gateway	Cloud Functions + Cloud Run
데이터 연동	S3 + DynamoDB	GCS + BigQuery

2. Google Cloud Vertex AI 시작

2.1 Vertex AI 소개

2.1.1 Vertex AI 란?

Vertex AI 는 Google Cloud 의 통합 AI/ML 플랫폼으로, 다양한 파운데이션 모델(FM)을 단일 API 로 제공하는 완전 관리형 서비스입니다. AWS Bedrock 이 '모델 슈퍼마켓'이라면, Vertex AI 는 '모델 슈퍼마켓 + ML 워크벤치'를 결합한 형태입니다.

Vertex AI 의 핵심 가치:

- 완전 관리형 (Fully Managed): 모델을 실행하기 위한 서버(인프라)를 전혀 관리할 필요가 없습니다. (Serverless)
- 단일 API: Gemini, Claude, Llama 등 어떤 모델을 사용하든 동일한 API 형식으로 호출할 수 있어, 모델 교체나 테스트가 매우 쉽습니다.
- 데이터 보안: 고객의 데이터는 외부 모델 학습에 절대 사용되지 않으며, GCP 프로젝트 내에서 안전하게 처리됩니다.
- BigQuery 연동: GCP 의 데이터 생태계(BigQuery, GCS, Dataflow 등)와 자연스럽게 통합됩니다.

2.1.2 Vertex AI 주요 기능

기능	설명	AWS Bedrock 대응
Model Garden	다양한 FM 을 탐색/비교/테스트할 수 있는 모델 카탈로그	Playground
Vertex AI Studio	프롬프트를 실험하고 모델 응답을 비교하는 웹 UI	Playground
Grounding / RAG API	기업 내부 문서를 AI 가 참조하여 답변 (RAG)	Knowledge Bases
Agent Builder	AI 가 실제 행동(Action)을 수행하게 만들	Bedrock Agents
Fine-tuning	기업 고유 데이터로 모델 특화	Fine-tuning
Safety Filters	AI 응답을 정책에 맞게 제어	Guardrails

2.1.3 Vertex AI 제공 모델

Vertex AI Model Garden 에서 제공하는 주요 모델 라인업입니다.

1. 텍스트 / 멀티모달 모델

개발사	모델명	주요 특징
Google	Gemini 2.0 Flash / Pro	Google 의 최신 멀티모달. 1M+ 토큰 컨텍스트, 코드/분석 등 범용
Anthropic	Claude 3.5 Sonnet / Haiku	복잡한 추론, 긴 문서 요약, 창의적 글쓰기에 탁월
Meta	Llama 3.1 (8B, 70B, 405B)	가장 강력한 오픈소스 모델, 빠른 응답 속도
Mistral AI	Mistral Large / Mixtral	MoE 기술로 비용 대비 성능 우수

2. 임베딩 모델 (Embedding Models)

임베딩이란 텍스트(단어, 문장)를 컴퓨터가 이해할 수 있는 숫자 벡터(Vector)로 변환하는 과정입니다. RAG 와 시맨틱 검색의 핵심 엔진입니다.

- Google: text-embedding-005 — 768 차원, 다국어 지원, Vertex AI 와 완벽 통합
- Google: textembedding-gecko — 경량 임베딩 모델, 빠른 처리 속도

3. 이미지 생성 모델

- Google: Imagen 3 — 텍스트 프롬프트로 고품질 이미지 생성
- Stability AI: Stable Diffusion (Model Garden 통해 배포 가능)

2.2 Vertex AI 사용하기

2.2.1 기본 호출 (Python SDK)

로컬 환경에서 Vertex AI 를 호출하는 기본 방법입니다. 상세한 코드는 실습 파일 01_basic_vertexai.py 를 참고하세요.

```
# 필수 설치
pip install google-cloud-aiplatform langchain-google-vertexai


# 기본 호출 예시
import vertexai
from vertexai.generative_models import GenerativeModel


vertexai.init(project="your-project-id", location="us-central1")
model = GenerativeModel("gemini-2.0-flash")
response = model.generate_content("Vertex AI 를 설명해주세요.")
```

```
print(response.text)
```

2.2.2 LangChain 연동

LangChain 을 통해 Vertex AI 모델을 호출하면, 프롬프트 템플릿, 체인, 출력 파서 등 편리한 기능을 활용할 수 있습니다. 상세한 코드는 실습 파일 01_basic_vertexai.py 의 LangChain 섹션을 참고하세요.

```
from langchain_google_vertexai import ChatVertexAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser


llm = ChatVertexAI(model_name="gemini-2.0-flash", temperature=0.2)
prompt = ChatPromptTemplate.from_messages([
    ("system", "너는 한국어를 잘하는 AI 조교야."),
    ("human", "{question}")
])
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"question": "Vertex AI 가 뭐야?"})
```

2.3 Gemini 모델 상세

2.3.1 Gemini 모델 라인업 비교

Vertex AI 의 핵심 모델인 Gemini 는 용도와 비용에 따라 여러 버전으로 제공됩니다. 각 모델의 특성을 이해하면 용도에 맞는 최적의 모델을 선택할 수 있습니다.

모델	컨텍스트	특징	적합한 용도
Gemini 2.5 Pro	1M 토큰	최상위 추론/코딩, 슬로우 응답	복잡한 분석, 코드 생성, 에이전트
Gemini 2.5 Flash	1M 토큰	빠른 응답, 사고(Thinking) 지원	대규모 처리, 일반 업무
Gemini 2.0 Flash	1M 토큰	가성비 우수, 빠른 속도	프로토타입, 비용 민감 서비스
Gemini 2.0 Flash Lite	1M 토큰	최저 비용, 초고속	대량 배치, 분류 작업

2.3.2 Gemini 가격 체계 (Vertex AI)

Vertex AI 에서 Gemini 모델을 사용할 때는 입력/출력 토큰 단위로 과금됩니다.
토큰(Token)은 모델이 텍스트를 처리하는 최소 단위로, 한국어 1 글자는 보통 1~3 토큰에 해당합니다.

모델	입력 (100 만 토큰당)	출력 (100 만 토큰당)	참고
Gemini 2.5 Pro	$1.25	$10.00	최고 성능, 추론 모델
Gemini 2.5 Flash	$0.15	$0.60	사고 토큰 별도 과금
Gemini 2.0 Flash	$0.10	$0.40	가성비 최고
Gemini 2.0 Flash Lite	$0.075	$0.30	최저 비용

비용 예시:

Gemini 2.0 Flash 로 한국어 500 자 입력(~1,000 토큰) + 1,000 자 출력(~2,000 토큰) 요청 시: 입력 $0.0001 + 출력 $0.0008 = 약 $0.001 (약 1.3 원). 하루 1,000 회 호출해도 약 1,300 원 수준입니다.

2.3.3 모델 파라미터 상세

Gemini 모델의 응답 스타일을 제어하는 핵심 파라미터입니다. 용도에 맞게 조절하면 보다 일관된 결과를 얻을 수 있습니다.

파라미터	범위	기본값	설명	권장 설정
temperature	0.0 ~ 2.0	1.0	높을수록 창의적, 낮을수록 일관된 응답	RAG/분석: 0.1~0.3, 창작: 0.7~1.0
top_p	0.0 ~ 1.0	0.95	누적 확률 기반 토큰 선택	대부분 0.8~0.95 권장
top_k	1 ~ 40	40	상위 K 개 토큰에서 선택	일반적으로 기본값 사용
max_output_tokens	1 ~ 65,536	모델별 상이	최대 출력 토큰 수	용도에 맞게 제한

temperature 값에 따른 응답 차이 예시:

프롬프트: "AI 를 한 단어로 설명해줘"

- temperature=0.0 → "혁신" (매번 같은 답변)
- temperature=0.5 → "지능" (비슷하지만 약간의 변화)
- temperature=1.5 → "변혁" / "무한" / "쳐니" (매번 다른 답변)

2.4 Gemini 멀티모달 활용

2.4.1 멀티모달이란?

Gemini 는 텍스트뿐만 아니라 이미지, 오디오, 비디오, PDF 등 다양한 형태의 입력을 동시에 처리할 수 있는 멀티모달(Multimodal) 모델입니다. 하나의 API 호출에 여러 형태의 입력을 함께 전달할 수 있어, 복합적인 작업을 효율적으로 처리합니다.

지원하는 입력 모달리티:

입력 유형	지원 형식	예시 활용
텍스트	일반 문자열	질문답변, 요약, 번역
이미지	JPEG, PNG, GIF, WebP	이미지 분석, OCR, 차트 해석
PDF	PDF 문서	문서 요약, 테이블 추출
오디오	MP3, WAV, FLAC	음성 전사, 회의록 요약
비디오	MP4, MOV, AVI	비디오 요약, 장면 분석

2.4.2 이미지 분석

Gemini 에게 이미지를 전달하는 방법은 3 가지입니다:

방법	설명	적합한 경우
GCS URI	gs://bucket/image.jpg 경로로 전달	GCP 환경, 대용량 파일
Base64 인코딩	이미지를 base64 로 변환하여 전달	로컬 파일, 20MB 이하
URL	https:// 공개 URL 로 전달	웹 이미지 바로 분석

코드 예시 (Vertex AI SDK):

```
from google import genai
from google.genai.types import HttpOptions, Part


client = genai.Client(http_options=HttpOptions(api_version="v1"))
response = client.models.generate_content(
    model="gemini-2.0-flash",
```

```
        contents=[
            "이 이미지에 뭐가 있나요?",
            Part.from_uri(
                file_uri="gs://cloud-samples-data/generative-ai/image/scones.jpg",
                mime_type="image/jpeg",
            ),
        ],
    )
    print(response.text)
```

상세한 실습 코드는 02b_multimodal.py 를 참고하세요.

2.4.3 PDF 문서 분석

Gemini 는 PDF 파일을 직접 입력으로 받아 요약, 번역, 테이블 추출 등을 수행할 수 있습니다. 별도의 OCR 라이브러리 없이 모델이 직접 문서를 이해합니다.

활용 예시:

- 계약서 PDF → 핵심 조항 추출
- 연구 논문 PDF → 3 줄 요약
- 송장 PDF → 항목별 금액 테이블 생성
- 외국어 문서 PDF → 한국어 번역

RAG(Chapter 4)에서는 PDF 를 청크로 분할하여 벡터 DB 에 저장했지만, 간단한 분석 작업이라면 Gemini 에 PDF 를 직접 전달하는 것이 더 효율적입니다.

2.4.4 복합 입력 (이미지 + 텍스트)

Gemini 의 가장 강력한 기능 중 하나는 여러 모달리티를 결합한 복합 입력입니다. 예를 들어:

- 제품 이미지 + "이 제품의 장단점을 분석해줘" → 시각적 분석 + 텍스트 보고서
- 설계도 이미지 + "이 설계도의 문제점을 찾아줘" → 기술적 리뷰
- 차트 이미지 + "이 차트의 트렌드를 분석해줘" → 데이터 분석

2.5 Safety Settings (안전 설정)

2.5.1 Safety Settings 란?

Gemini 는 유해한 콘텐츠를 자동으로 필터링하는 안전 시스템을 갖추고 있습니다. 개발자는 애플리케이션의 용도에 맞게 이 필터의 강도를 조절할 수 있습니다. AWS Bedrock 의 Guardrails 와 유사한 기능입니다.

2.5.2 유해 카테고리

카테고리	영문명	설명
헤이트 스피치	HARM_CATEGORY_HATE_SPEECH	인종, 성별, 종교 등에 대한 혐오 발언
괴롭힘	HARM_CATEGORY_HARASSMENT	개인을 대상으로 한 괴롭힘/위협
성적 콘텐츠	HARM_CATEGORY_SEXUALLY_EXPLICIT	성적으로 노골적인 콘텐츠
위험한 콘텐츠	HARM_CATEGORY_DANGEROUS_CONTENT	자해/타해를 조장하는 콘텐츠

2.5.3 차단 임계값 (Threshold)

임계값	설명	엄격도
BLOCK_LOW_AND_ABOVE	LOW 이상 모두 차단	가장 엄격
BLOCK_MEDIUM_AND_ABOVE	MEDIUM 이상 차단 (기본값)	일반적
BLOCK_ONLY_HIGH	HIGH 만 차단	느슨
BLOCK_NONE	차단 없음 (메타데이터는 반환)	차단 없음
OFF	필터링 완전 비활성화	비활성화

코드 예시:

```
from vertexai.generative_models import (
    GenerativeModel, SafetySetting, HarmCategory, HarmBlockThreshold
)


safety_settings = [
    SafetySetting(
        category=HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
        threshold=HarmBlockThreshold.BLOCK_ONLY_HIGH,
    ),
```

```
        SafetySetting(
            category=HarmCategory.HARM_CATEGORY_HARASSMENT,
            threshold=HarmBlockThreshold.BLOCK_ONLY_HIGH,
        ),
    ]


    model = GenerativeModel("gemini-2.0-flash")
    response = model.generate_content("...", safety_settings=safety_settings)
```

상세한 실습 코드는 02b_multimodal.py 의 실습 D 를 참고하세요.

3. 프롬프트 엔지니어링 (Prompt Engineering)

3.1 프롬프트 엔지니어링의 개념과 중요성

① 개념 정의

프롬프트 엔지니어링(Prompt Engineering)은 AI(특히 LLM)가 사용자의 의도를 정확히 이해하고 원하는 결과를 생성하도록 입력문(프롬프트)을 설계하는 기술입니다. 이는 플랫폼(AWS/GCP)에 관계없이 동일하게 적용되는 보편적인 기술입니다.

② 프롬프트 엔지니어링이 중요한 이유

구분	설명
효율성 향상	적절한 프롬프트는 반복 수정 없이 원하는 결과를 빠르게 얻을 수 있음
정확도 향상	불필요한 오답이나 비논리적 결과를 줄임
비용 절감	API 사용 시 토큰 소모량 감소 → 모델 호출 비용 절감
응용 확장성	다양한 업무 자동화, 분석, 생성형 AI 응용에 직접 활용 가능

3.2 효과적인 프롬프트 작성 방법

① 기본 원칙 (CLEAR)

원칙	의미	예시
C (Clear)	명확한 지시	"요약해줘" → "이 문단을 3 줄로 요약해줘."
L (Logical)	논리적 순서	"(1) 개념 (2) 예시 (3) 결론 순서로 설명해줘."
E (Explicit)	구체적인 조건	"300 자 이내로, 초보자도 이해할 수 있게 작성해줘."
A (Adaptive)	대상 맞춤형	"고등학생 수준으로 쉽게 설명해줘."
R (Role-based)	역할 부여	"너는 Python 전문가야. 아래 코드를 리뷰해줘."

② 프롬프트 구조 설계 4 단계

- 역할(Role) 부여 → 모델에게 전문가 정체성 부여. 예: "너는 데이터 분석 전문가야."
- 목표(Objective) 명시 → 수행해야 할 구체적 작업. 예: "다음 CSV 데이터를 요약해줘."
- 형식(Format) 지정 → 출력 형태 명시. 예: "표 형식으로 정리해줘."
- 제약조건(Constraints) → 제한 사항 설정. 예: "150 자 이내, 기술 용어는 영어 그대로."

③ 고급 기법

기법	설명	예시
Few-shot Prompting	예시를 몇 개 제시해 모델이 패턴을 학습하도록 함	"예시 A, B 처럼 작성해줘."
Chain of Thought (CoT)	모델이 단계별로 사고 과정을 전개하도록 유도	"단계를 나눠서 생각해보고 답을 도출해줘."
Self-Consistency	여러 번의 응답 중 일관된 결과를 선택	"3 가지 답변을 생성하고, 가장 논리적인 것을 선택해."
ReAct	추론(Reasoning) + 행동(Action) 결합	"단서를 바탕으로 추론 후, 필요한 정보를 검색하라."

실습 코드는 02_prompt_engineering.py 를 참고하세요. Vertex AI 에서 각 기법을 직접 실행해볼 수 있습니다.

3.3 Grounding 개념

3.3.1 Grounding 이란?

Grounding 은 LLM 의 응답을 검증 가능한 외부 데이터에 기반하여 생성하는 기법입니다. 모델이 자체 학습 데이터만으로 응답하면 할루시네이션(Hallucination)이 발생할 수 있습니다. Grounding 을 사용하면 모델이 실시간 검색 결과나 기업 내부 문서를 참조하여 정확하고 최신의 정보를 제공합니다.

AWS 비교: AWS Bedrock 에서는 Knowledge Bases 를 통해 RAG 방식으로 외부 데이터를 연결합니다. Vertex AI 의 Grounding 은 이와 유사하지만, Google Search 를 통한 웹 검색 결과를 직접 모델 응답에 반영할 수 있다는 점이 차별화 포인트입니다.

3.3.2 Grounding 이 필요한 이유

문제	Grounding 없이	Grounding 사용 시
할루시네이션	모델이 그럴듯한 정보를 생성	검증된 출처 기반 응답
정보 노후화	학습 데이터 시점의 정보만 반영	실시간 검색으로 최신 정보 반영
출처 부재	응답의 근거를 확인할 수 없음	출처 URL, 신뢰도 점수 함께 제공

> **핵심:** Grounding 은 Chapter 4 의 RAG 와 목적이 비슷하지만 접근 방식이 다릅니다. RAG 는 직접 문서를 임베딩→검색하고, Grounding 은 Google Search 또는 Vertex AI Search 엔진에 검색을 위임합니다.

3.4 Google Search Grounding

3.4.1 개요

Google Search Grounding 은 Gemini 모델이 응답을 생성하기 전에 Google 검색을 수행하여 최신 정보를 참조하는 기능입니다. 모델의 학습 데이터에 포함되지 않은 최신 정보(주가, 날씨, 뉴스 등)를 포함한 응답을 생성할 수 있습니다.

동작 흐름:

1. 사용자가 질문을 입력합니다 ("2025 년 한국 경제 성장률은?")
2. Gemini 가 자동으로 Google Search 를 수행합니다.
3. 검색 결과를 컨텍스트로 활용하여 응답을 생성합니다.
4. 응답과 함께 출처 URL 과 Grounding 메타데이터가 반환됩니다.

3.4.2 Google Search Grounding 유형

항목	Google Search Grounding	Enterprise Web Grounding
대상 데이터	공개 웹 전체 (Google 검색)	기업이 지정한 웹사이트/도메인
용도	일반 최신 정보 조회	특정 도메인의 정보만 참조
설정	google_search Tool 추가	Vertex AI Search 데이터스토어 연결
비용	검색당 과금 (Dynamic Retrieval)	Vertex AI Search 요금제 적용

3.4.3 동적 검색 (Dynamic Retrieval)

Dynamic Retrieval 은 모델이 모든 질문에 대해 검색을 수행하는 것이 아니라, 필요한 경우에만 검색을 수행하도록 제어하는 기능입니다. 임계값(threshold)을 설정하여 검색 빈도를 조절합니다.

임계값	검색 빈도	비용	적합한 상황
0.0	항상 검색	높음	실시간 정보 필수
0.3	자주 검색	중간	뉴스/이벤트 관련 서비스
0.7	가끔 검색	낮음	일반 챗봇 + 필요시 검색
1.0	검색 안 함 (설정무효)	없음	Grounding 불필요 작업

코드 예시 (Vertex AI SDK):

```
from google import genai
from google.genai.types import (
    HttpOptions, Tool, GoogleSearch,
    DynamicRetrievalConfig, GenerateContentConfig
)


client = genai.Client(
    http_options=HttpOptions(api_version="v1")
)


google_search_tool = Tool(
    google_search=GoogleSearch(
        dynamic_retrieval_config=DynamicRetrievalConfig(
```

```
            dynamic_threshold=0.3  # 0.0~1.0
        )
    )
)


response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="오늘 한국 주요 뉴스 3 가지를 알려줘",
    config=GenerateContentConfig(
        tools=[google_search_tool],
    ),
)
```

상세한 실습 코드는 03b_grounding.py 를 참고하세요.

3.4.4 Grounding 응답 메타데이터

Grounding 을 사용하면 응답과 함께 다음과 같은 메타데이터가 반환됩니다. 이 메타데이터를 활용하면 응답의 신뢰성을 검증하고 사용자에게 출처를 제공할 수 있습니다.

메타데이터 필드	설명
grounding_metadata	전체 Grounding 결과 컨테이너
search_entry_point	검색 결과 페이지 URL (Google Search 엔트리 포인트)
grounding_chunks	참조한 웹 페이지들의 목록 (URL, 제목)
grounding_supports	응답 텍스트의 어느 부분이 어떤 출처에 기반하는지 매핑
web_search_queries	모델이 실제로 수행한 검색 쿼리 목록

메타데이터 파싱 예시:

```
# 응답의 Grounding 메타데이터 확인
candidate = response.candidates[0]
metadata = candidate.grounding_metadata


# 모델이 수행한 검색 쿼리
for query in metadata.web_search_queries:
    print(f"검색 쿼리: {query}")


# 참조한 출처 확인
for chunk in metadata.grounding_chunks:
    web = chunk.web
    print(f"출처: {web.title} - {web.uri}")
```

TIP: 서비스에서 Grounding 을 사용할 때는 grounding_supports 를 활용하여 응답의 각 문장에 출처 링크를 추가하면 사용자 신뢰를 높일 수 있습니다.

3.5 Vertex AI Search Grounding (기업 데이터)

3.5.1 개요

Google Search Grounding 이 공개 웹 정보를 활용하는 반면, Vertex AI Search Grounding 은 기업 내부 문서를 대상으로 합니다. 사내 규정, 기술 문서, FAQ 등을 데이터스토어에 업로드하면 Gemini 가 해당 문서를 참조하여 응답합니다.

3.5.2 Vertex AI Search 데이터스토어 생성 흐름

1 단계: 데이터스토어 생성

GCP Console > Vertex AI Search > 데이터스토어 만들기에서 새 데이터스토어를 생성합니다.

2 단계: 데이터 소스 연결

데이터 소스	설명	적합한 경우
Cloud Storage (GCS)	GCS 버킷의 PDF, HTML, TXT 등	기술 문서, 매뉴얼
Google Drive	Drive 폴더의 문서 자동 동기	사내 공유 문서
웹사이트 URL	지정 URL 을 크롤링하여 인덱싱	공개 도움말, FAQ 페이지
BigQuery	BigQuery 테이블 데이터 연결	구조화된 데이터 검색

3 단계: Gemini 에서 Grounding 사용

데이터스토어 생성 후, 코드에서 Vertex AI Search 를 Grounding 소스로 지정합니다.

```
from google import genai
from google.genai.types import (
    HttpOptions, Tool, VertexAISearch,
    GenerateContentConfig
)


client = genai.Client(
    http_options=HttpOptions(api_version="v1")
)


datastore_path = (
    "projects/PROJECT_ID"
```

```
    "/locations/global"
    "/collections/default_collection"
    "/dataStores/DATASTORE_ID"
)


grounding_tool = Tool(
    retrieval=VertexAISearch(datastore=datastore_path)
)


response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="사내 휴가 규정에 대해 알려줘",
    config=GenerateContentConfig(
        tools=[grounding_tool],
    ),
)
```

3.5.3 Google Search vs Vertex AI Search 비교

항목	Google Search Grounding	Vertex AI Search Grounding
데이터 범위	공개 웹 전체	기업 내부 문서
데이터 업데이트	Google 이 자동 관리	직접 업로드/동기 필요
보안	공개 정보만 참조	기업 데이터 격리 보장
적합한 용도	일반 정보 챗봇, 뉴스 기반 응답	사내 고객센터, 기술지원 봇
설정 복잡도	간단 (Tool 추가만 하면 됨)	중간 (데이터스토어 생성 필요)
RAG Engine 비교	웹 검색 기반, 커스터마이징 제한	문서 기반, 중간 수준 커스터마이징

3.5.4 LangChain 에서 Grounding 사용

LangChain 의 ChatVertexAI 를 통해서도 Google Search Grounding 을 활용할 수 있습니다.
LangChain 의 Tool Calling 기능을 통해 Grounding 을 도구(Tool)로 등록하여 사용합니다.

코드 예시:

```
from langchain_google_vertexai import ChatVertexAI


llm = ChatVertexAI(
    model="gemini-2.0-flash",
    temperature=0,
)


# Google Search Grounding 활성화
```

```
from langchain_google_community import (
    VertexAISearchRetriever
)
# 또는 ChatVertexAI 의 bind_tools 로
# google_search 도구를 바인딩
```

상세한 실습 코드는 03b_grounding.py 의 실습 E 를 참고하세요.

> **Grounding vs RAG 선택 가이드:** 최신 공개 정보가 필요하면 Google Search Grounding, 기업 내부 문서를 참조해야 하면 Vertex AI Search Grounding 또는 RAG(Chapter 4)를 사용하세요. RAG Engine(Phase 3)은 더 정밀한 컨트롤이 필요할 때 적합합니다.

4. Vertex AI 로 RAG 구현

4.1 RAG 개념

RAG(Retrieval-Augmented Generation)란?

RAG 는 AI 모델이 답변을 생성하기 전에 외부 지식 소스(문서, DB 등)에서 관련 정보를 먼저 검색(Retrieve)하고, 이를 근거로 답변을 생성(Generate)하는 기법입니다.

RAG 가 필요한 이유:

- Hallucination 방지: 모델이 알지 못하는 정보를 지어내는 것을 방지
- 최신 정보 반영: 모델의 학습 데이터 컷오프 이후의 정보도 참조 가능
- 기업 내부 데이터 활용: 내부 문서, 규정, FAQ 등을 AI 가 참조하여 답변

RAG 처리 흐름:

- 문서 수집 → PDF, DOCX, HTML 등 문서를 수집
- 문서 분할 (Chunking) → 문서를 적절한 크기로 분할
- 임베딩 변환 → 분할된 텍스트를 벡터로 변환
- 벡터 DB 저장 → ChromaDB, Pinecone, Weaviate 등에 저장
- 질문 → 사용자 질문을 임베딩하여 유사 문서 검색
- 응답 생성 → 검색된 문서 + 질문을 LLM 에 전달하여 답변 생성

4.2 벡터 데이터베이스

벡터 DB 는 텍스트를 숫자 벡터로 변환하여 저장하고, 유사도 기반으로 검색하는 데이터베이스입니다.

벡터 DB	특징	적합한 경우
ChromaDB	로컬 실행, 설치 간편, Python 네이티브	프로토타입/학습 용도
Pinecone	클라우드 관리형, 대규모 처리	프로덕션 환경
Weaviate	오픈소스, 하이브리드 검색	복잡한 검색 요구사항
AlloyDB (GCP)	GCP 관리형, pgvector 통합	GCP 에코시스템 연동

4.3 LangChain 으로 RAG 구현 (로컬)

LangChain + Vertex AI + ChromaDB 를 사용하여 로컬에서 RAG 를 구현합니다. 상세한 코드는 실습 파일 03_rag_vertexai.py 를 참고하세요.

필요 라이브러리:

```
pip install langchain langchain-google-vertexai langchain-chroma
pip install chromadb pypdf sentence-transformers
```

구현 흐름 요약:

- PDF 문서 로드 → PyPDFLoader 로 문서 읽기
- 문서 분할 → RecursiveCharacterTextSplitter 로 청크 분할
- 임베딩 + 벡터 DB 저장 → VertexAIEmbeddings + ChromaDB
- RAG 체인 구성 → RetrievalQA 체인으로 질의응답

4.4 Vertex AI RAG Engine

4.4.1 RAG Engine 이란?

Vertex AI RAG Engine 은 Google Cloud 에서 제공하는 관리형 RAG(Retrieval-Augmented Generation) 서비스입니다. Chapter 4 에서 LangChain + ChromaDB 로 직접 RAG 를 구현했다면, RAG Engine 은 문서 임포트, 임베딩, 청킹, 벡터 저장, 검색까지의 전체 파이프라인을 Google 이 관리해주는 서비스입니다.

AWS 비교: AWS Bedrock 의 Knowledge Bases 와 유사합니다. Bedrock Knowledge Bases 가 S3 문서를 자동으로 임베딩/인덱싱하는 것처럼, RAG Engine 은 GCS, Google Drive 등의 문서를 자동으로 처리합니다. 다만 RAG Engine 은 Gemini 모델과 직접 통합되어 별도 오케스트레이션 없이 RAG 응답을 생성할 수 있습니다.

4.4.2 RAG Engine 아키텍처

RAG Engine 은 다음과 같은 구성 요소로 이루어져 있습니다:

구성 요소	설명	역할
RAG Corpus	문서 컬렉션의 최상위 컨테이너	여러 파일을 묶어 하나의 검색 대상으로 관리
RAG Files	Corpus 에 임포트된 개별 문서	자동으로 청킹 → 임베딩 → 인덱싱 처리
Embedding	Vertex AI Embedding 모델 (text-embedding-005)	문서와 쿼리를 벡터로 변환
Vector DB	내부 벡터 데이터베이스 (또는 Weaviate, Pinecone 연동)	벡터 유사도 검색 수행
Retrieval API	retrieveContexts API	쿼리와 관련된 문서 청크를 반환

RAG Engine 의 동작 흐름은 다음과 같습니다:

1. RAG Corpus 생성: 프로젝트 내에 문서 컬렉션을 생성합니다.

2. 파일 임포트: GCS, Google Drive, 또는 웹페이지에서 문서를 임포트합니다.

3. 자동 처리: RAG Engine 이 문서를 청킹하고, 임베딩을 생성하고, 벡터 DB 에 저장합니다.

4. 컨텍스트 검색: 사용자 쿼리를 임베딩하여 관련 문서 청크를 검색합니다.

5. 응답 생성: Gemini 모델이 검색된 컨텍스트를 참조하여 응답을 생성합니다.

4.4.3 지원 데이터 소스

RAG Engine 은 다양한 데이터 소스에서 문서를 임포트할 수 있습니다:

데이터 소스	설명	지원 파일 형식
Cloud Storage (GCS)	GCS 버킷의 파일 임포트	PDF, TXT, HTML, DOCX, PPTX, XLSX
Google Drive	Drive 폴더의 문서 자동 동기	Google Docs, PDF, TXT 등
웹페이지 URL	지정 URL 을 크롤링하여 인덱싱	HTML 페이지
Slack	Slack 채널 메시지 인덱싱	텍스트 메시지
Jira	Jira 이슈/티켓 인덱싱	이슈 내용

TIP: RAG Engine 은 문서를 임포트하면 자동으로 청킹과 임베딩을 처리합니다. 별도로 LangChain 의 TextSplitter 나 임베딩 모델을 직접 호출할 필요가 없습니다.

4.4.4 RAG Engine 코드 예시

Vertex AI Python SDK 를 사용하여 RAG Engine 을 설정하고 쿼리하는 기본 코드 구조입니다:

Corpus 생성 및 파일 임포트:

```
from vertexai import rag
import vertexai


# 초기화
vertexai.init(project="PROJECT_ID", location="us-central1")


# RAG Corpus 생성
corpus = rag.create_corpus(
    display_name="my-rag-corpus",
    description="기업 내부 문서 컬렉션"
)
print(f"Corpus 생성: {corpus.name}")


# GCS 에서 파일 임포트
rag.import_files(
    corpus.name,
    paths=["gs://my-bucket/documents/"],
    chunk_size=512,
    chunk_overlap=100,
)
```

컨텍스트 검색 및 응답 생성:

```
# 컨텍스트 검색 (검색만 수행)
```

```
response = rag.retrieval_query(
    rag_resources=[
        rag.RagResource(rag_corpus=corpus.name)
    ],
    text="연차 휴가 규정은?",
    similarity_top_k=5,
)

for context in response.contexts.contexts:
    print(f"출처: {context.source_uri}")
    print(f"내용: {context.text[:200]}")

# Gemini + RAG Tool 로 응답 생성
from vertexai.generative_models import GenerativeModel, Tool

rag_tool = Tool.from_retrieval(
    retrieval=rag.Retrieval(
        source=rag.VertexRagStore(
            rag_resources=[
                rag.RagResource(rag_corpus=corpus.name)
            ],
            similarity_top_k=5,
        ),
    )
)

model = GenerativeModel("gemini-2.0-flash", tools=[rag_tool])
response = model.generate_content("연차 휴가 규정에 대해 알려줘")
print(response.text)
```

상세한 실습 코드는 03c_rag_engine.py 를 참고하세요.

4.5 RAG 구현 방식 3 종 비교

4.5.1 LangChain 직접 구현 vs RAG Engine vs Vertex AI Search

Vertex AI 에서 RAG 를 구현하는 방법은 크게 세 가지입니다. 각 방식의 특성과 적합한 사용 시나리오를 비교합니다.

항목	LangChain 직접 구현	RAG Engine	Vertex AI Search
구현 방식	LangChain + 벡터 DB (ChromaDB, pgvector 등)	Vertex AI 관리형 API	Vertex AI Search 데이터스토어
청킹 제어	완전한 제어 (크기, 오버랩, 전략)	청크 크기/오버랩 설정 가능	Google 이 자동 처리
임베딩 모델	직접 선택 (Vertex AI, OpenAI 등)	Vertex AI Embedding 자동 사용	Google 내부 모델 사용
벡터 DB	직접 선택 (ChromaDB, pgvector, Pinecone 등)	관리형 (Weaviate, Pinecone 연동 가능)	Google 관리형
커스터마이징	높음 (Reranker, 필터, 하이브리드 검색)	중간 (파라미터 조정 가능)	낮음 (Google 이 최적화)
관리 부담	높음 (직접 인프라 관리)	낮음 (Google 관리)	매우 낮음 (Fully Managed)
비용	인프라 비용 + API 비용	API 호출 비용만	Vertex AI Search 요금제
적합한 사용 시나리오	정밀한 제어가 필요한 프로덕션 환경	빠른 프로토타입 + 중간 수준 커스터마이징	빠른 도입 + 최소 관리

핵심: PoC 나 빠른 프로토타입에는 RAG Engine 이 적합하고, 정밀한 검색 품질 튜닝이 필요한 프로덕션 환경에서는 LangChain + pgvector(Chapter 6) 조합을 권장합니다. 문서 검색만 필요하고 관리 부담을 최소화하려면 Vertex AI Search 를 사용하세요.

4.5.2 Grounding vs RAG 선택 가이드

Phase 2 에서 학습한 Grounding 과 RAG 는 모두 외부 데이터를 활용하여 모델 응답의 품질을 높이는 기법이지만, 적용 시나리오가 다릅니다:

항목	Grounding	RAG (Engine / 직접 구현)
데이터 소스	공개 웹 (Google Search) 또는 데이터스토어	기업 내부 문서 (PDF, DB 등)
검색 방식	Google 이 검색 수행	벡터 유사도 검색
데이터 관리	별도 관리 불필요	문서 업로드/동기 필요
최신성	실시간 웹 검색	임포트된 문서 기준
적합한 질문	"오늘 날씨는?", "최신 뉴스"	"사내 휴가 규정은?", "제품 스펙"
결합 사용	Grounding + RAG 를 함께 사용 가능	공개 정보 + 사내 문서 동시 참조

4.6 RAG 품질 향상 기법

4.6.1 청킹 전략 (Chunking Strategy)

청킹은 RAG 의 품질을 결정하는 가장 중요한 요소입니다. 문서를 어떻게 분할하느냐에 따라 검색 정확도가 크게 달라집니다.

청킹 방식	설명	장점	단점
고정 크기	일정 토큰/문자 수로 분할 (512~1024 토큰)	구현 간단, 예측 가능한 크기	문맥이 중간에 끊길 수 있음
오버랩 청킹	청크 간 일부 텍스트를 중복 (50~200 토큰)	문맥 손실 보완	저장 공간 증가
시맨틱 분할	의미 단위로 분할 (단락, 섹션 기준)	문맥 보존 우수	구현 복잡, 처리 시간 김
문서 구조 기반	Markdown 헤더, HTML 태그 등으로 분할	문서 구조 활용	구조화되지 않은 문서에는 부적합

권장 설정: RAG Engine 사용 시 chunk_size=512, chunk_overlap=100 으로 시작하고, 검색 결과를 평가하며 조정하세요. 문서가 길고 구조화된 경우 1024~2048 로 늘리는 것도 효과적입니다.

4.6.2 리랭킹 (Reranking)

리랭킹은 벡터 검색으로 가져온 초기 결과를 다시 정밀하게 순위를 매기는 기법입니다. 벡터 유사도만으로는 질문의 의도와 정확히 일치하지 않는 결과가 상위에 오는 경우가 있습니다.

리랭킹의 동작 방식:

1. 벡터 검색으로 상위 K 개 문서를 가져옵니다 (예: top_k=20).

2. Reranker 모델이 쿼리와 각 문서의 관련성을 교차 분석합니다.

3. 새로운 점수 기준으로 상위 N 개를 선택합니다 (예: top_n=5).

코드 예시 (LangChain Reranker):

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain_google_community import VertexAIRank


# Vertex AI Rank 를 리랭커로 사용
compressor = VertexAIRank(
    model_name="semantic-ranker-512@latest",
    top_n=5,
)
```

```
# 기존 retriever에 리랭킹 추가
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vector_store.as_retriever(
        search_kwargs={"k": 20}
    ),
)

# 검색 실행 (자동으로 리랭킹 적용)
results = compression_retriever.invoke("연차 휴가 규정")
```

4.6.3 하이브리드 검색 (Hybrid Search)

하이브리드 검색은 벡터 유사도 검색과 키워드(문자열) 검색을 결합하여 검색 품질을 높이는 방법입니다. 벡터 검색은 의미적 유사성에 강하지만, 정확한 키워드 매칭에는 약할 수 있습니다.

검색 방식	강점	약점
벡터 검색 (Dense)	의미적으로 유사한 문서 발견	정확한 용어/코드 검색 어려움
키워드 검색 (Sparse)	정확한 용어 매칭 (BM25 등)	동의어/유사 표현 인식 불가
하이브리드 검색	두 방식의 장점 결합	구현 복잡도 증가

예시: "휴가 신청서 양식 번호 HR-2024-001"라는 쿼리에서:

- 벡터 검색: "휴가 신청"과 의미적으로 유사한 문서를 찾음
- 키워드 검색: 정확히 "HR-2024-001"이 포함된 문서를 찾음
- 하이브리드: 두 결과를 결합하여 가장 관련성 높은 문서를 상위에 배치

> **TIP:** pgvector를 사용하면 SQL WHERE 절과 벡터 유사도 검색을 결합한 하이브리드 검색을 자연스럽게 구현할 수 있습니다. 자세한 내용은 Chapter 6의 DB 연동을 참고하세요.

4.6.4 RAG 품질 평가

RAG 시스템의 품질을 평가하기 위해 다음 지표를 활용할 수 있습니다:

평가 지표	설명	측정 방법
Retrieval Precision	검색된 문서 중 관련 문서의 비율	검색 결과를 수동으로 평가

Retrieval Recall	전체 관련 문서 중 검색된 비율	정답 문서 세트 대비 비교
Answer Accuracy	생성된 답변의 정확성	LLM 기반 자동 평가 또는 사람 평가
Faithfulness	답변이 검색된 문서에 근거하는지	RAGAS 등 평가 프레임워크 활용

Grounding 연계: Chapter 3.3 에서 학습한 Grounding 의 grounding_supports 메타데이터는 Faithfulness 평가와 유사한 역할을 합니다. 응답의 각 부분이 어떤 출처에 기반하는지 확인할 수 있습니다.

5. Cloud Functions + GCS 활용

AWS 에서 Lambda + S3 를 사용하는 것처럼, GCP 에서는 Cloud Functions + Google Cloud Storage(GCS)를 사용하여 서버리스 백엔드 API 를 구축합니다.

AWS	GCP	역할
Lambda	Cloud Functions (2nd gen)	서버리스 함수 실행
API Gateway	Cloud Functions HTTP 트리거	HTTP 엔드포인트
S3	Google Cloud Storage (GCS)	파일/배포 패키지 저장
IAM Role	Service Account	권한 관리

5.1 Cloud Functions 으로 Vertex AI 백엔드 API 구축

상세한 코드는 실습 파일 04_cloud_function/ 폴더를 참고하세요.

(1) 프로젝트 구조

```
cloud_function/
  main.py              # Cloud Function 핸들러
  requirements.txt     # 필요 라이브러리
```

(2) 배포 방법

A. gcloud CLI 로 배포:

```
gcloud functions deploy vertex-ai-api \
```

```
  --gen2 \
  --runtime python312 \
  --trigger-http \
  --allow-unauthenticated \
  --entry-point handle_request \
  --region us-central1 \
  --source .
```

B. 배포 후 테스트:

```
curl -X POST https://<FUNCTION_URL> \
  -H "Content-Type: application/json" \
  -d '{"question": "Vertex AI 가 뭐야?"}'
```

5.2 GCS 기반 배포

Cloud Functions 의 소스 코드가 큰 경우 GCS 버킷에 업로드하여 배포할 수 있습니다.

```
# GCS 에 소스 업로드
gsutil cp -r ./cloud_function gs://my-bucket/cloud_function


# GCS 에서 배포
gcloud functions deploy vertex-ai-api \
  --gen2 \
  --runtime python312 \
  --trigger-http \
  --source gs://my-bucket/cloud_function \
  --entry-point handle_request
```

5.3 로컬에서 테스트

배포된 Cloud Function 을 로컬 Python 에서 호출하는 방법입니다. 실습 파일 05_test_cloud_function.py 를 참고하세요.

```
import requests
```

```
FUNCTION_URL = "https://<YOUR_FUNCTION_URL>"
resp = requests.post(
    FUNCTION_URL, json={"question": "Vertex AI 가 뭐야?"}
)
print("답변:", resp.json()["answer"])
```

6. 데이터베이스 연동

Chapter 4 에서 LangChain + ChromaDB 로 RAG 를 구현하고, RAG Engine 으로 관리형 RAG 를 학습했습니다. 이번 Chapter 에서는 프로덕션 환경에서 가장 많이 사용되는 패턴인 데이터베이스 기반 벡터 검색을 다룹니다. PostgreSQL 의 pgvector 확장을 활용하여 벡터 데이터를 저장하고 검색하며, BigQuery 에서 Vertex AI 를 직접 호출하는 방법까지 학습합니다.

AWS 비교: AWS 에서는 Amazon RDS for PostgreSQL + pgvector, Amazon Aurora + pgvector, 또는 Amazon OpenSearch 를 벡터 검색에 사용합니다. GCP 에서는 Cloud SQL for PostgreSQL + pgvector 와 AlloyDB + pgvector 가 대응됩니다. BigQuery ML 의 ML.GENERATE_TEXT 는 Amazon Bedrock 의 SQL 통합과 유사한 개념입니다.

6.0 GCP 데이터베이스 서비스 개요

GCP 는 다양한 데이터베이스 서비스를 제공하며, AI/ML 워크로드에 따라 적합한 선택이 달라집니다. 벡터 검색이 가능한 주요 서비스를 정리합니다:

서비스	설명	벡터 검색	AWS 대응
Cloud SQL for PostgreSQL	Google 관리형 PostgreSQL	pgvector 확장 지원	Amazon RDS for PostgreSQL
AlloyDB for PostgreSQL	Google 고성능 PostgreSQL 호환 DB	pgvector + 최적화된 벡터 엔진	Amazon Aurora PostgreSQL
BigQuery	서버리스 데이터 웨어하우스	VECTOR_SEARCH 함수 지원	Amazon Redshift
Firestore	서버리스 NoSQL 문서 DB	벡터 검색 프리뷰 지원	Amazon DynamoDB
Memorystore for	관리형 Redis	벡터 유사도 검색	Amazon

Redis		지원	ElastiCache

AlloyDB vs Cloud SQL 비교

AlloyDB 와 Cloud SQL 은 모두 PostgreSQL 호환 관리형 데이터베이스이지만, 성능과 비용 측면에서 차이가 있습니다:

항목	**Cloud SQL for PostgreSQL**	**AlloyDB for PostgreSQL**
성능	표준 PostgreSQL 성능	트랜잭션 4 배, 분석 쿼리 100 배 빠름
벡터 검색	pgvector 확장 지원	pgvector + Google 최적화 벡터 엔진 (ScaNN)
비용	월 ~$50 부터 (소규모)	월 ~$300 부터 (최소 구성)
스토리지	표준 SSD/HDD	분산 스토리지 (자동 샤딩)
적합한 사용	개발/테스트, 소규모 프로덕션	대규모 프로덕션, 고성능 RAG
권장 시나리오	학습/PoC 환경, 비용 절감 우선	업무 크리티컬 RAG, 10 만+ 문서

TIP: 이 교육자료에서는 비용 효율적인 Cloud SQL for PostgreSQL + pgvector 를 사용합니다. 로컬 환경에서는 Docker 로 PostgreSQL + pgvector 를 구성하여 동일한 코드로 실습할 수 있습니다.

6.1 pgvector 개념

pgvector 는 PostgreSQL 에 벡터 데이터 타입과 벡터 유사도 검색 기능을 추가하는 오픈소스 확장(Extension)입니다. AI 임베딩 벡터를 SQL 테이블에 저장하고, SQL 쿼리로 벡터 유사도 검색을 수행할 수 있습니다. ChromaDB 같은 전용 벡터 DB 와 달리, 기존 PostgreSQL 인프라에 벡터 기능을 추가하는 방식이므로 기존 데이터와 벡터 데이터를 하나의 DB 에서 통합 관리할 수 있습니다.

pgvector 가 프로덕션에서 선호되는 이유

트랜잭션 + 벡터 통합: 기존 비즈니스 데이터(SQL)와 벡터 데이터를 하나의 DB 에서 관리합니다. 별도의 벡터 DB 를 운영할 필요가 없어 인프라 복잡도가 줄어듭니다.

SQL 기반 하이브리드 검색: SQL WHERE 절과 벡터 검색을 하나의 쿼리로 결합할 수 있습니다. 예: "가격 5 만원 이하 + 설명이 유사한 상품"

PostgreSQL 생태계 활용: 백업, 복제, 모니터링, 보안 등 PostgreSQL 의 엔터프라이즈 기능을 그대로 사용할 수 있습니다.

GCP 통합: Cloud SQL, AlloyDB 모두 pgvector 를 공식 지원하며, Cloud SQL Connector 로 안전한 연결이 가능합니다.

벡터 인덱스 종류

벡터 데이터가 많아지면 전체 스캔(Exact Search)은 느려집니다. 인덱스를 생성하면 근사 최근접 이웃(ANN) 검색으로 속도를 크게 높일 수 있습니다:

인덱스	알고리즘	장점	단점	권장 시나리오
IVFFlat	클러스터 기반 역색인	빌드 빠름, 메모리 적음	정확도 다소 낮음, 빌드 전 데이터 필요	10 만 건 미만, 빠른 초기 구축
HNSW	그래프 기반 탐색	높은 정확도 (recall)	빌드 느림, 메모리 많이 사용	10 만 건 이상, 높은 검색 품질 필요

인덱스 생성 SQL 예시:

```
-- IVFFlat 인덱스 (lists = sqrt(데이터 수) 권장)
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- HNSW 인덱스 (더 높은 정확도)
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

거리 함수 (Distance Functions)

pgvector 는 세 가지 거리 함수를 지원합니다. 임베딩 모델에 따라 적합한 함수가 다릅니다:

거리 함수	SQL 연산자	설명	Vertex AI Embedding 권장
L2 (유클리드)	<->	벡터 간 직선 거리 (값이 작을수록 유사)	일반적 사용
코사인 거리	<=>	방향 유사도 (1 - cosine similarity)	권장 (텍스트 임베딩에 최적)
내적 (Inner Product)	<#>	벡터의 내적 (음수, 값이 작을수록 유사)	정규화된 벡터에 사용

TIP: Vertex AI 의 text-embedding-005 모델은 코사인 유사도에 최적화되어 있으므로, pgvector 에서 <=> 연산자를 사용하는 것을 권장합니다.

6.2 Cloud SQL + pgvector 로 RAG 구현

아키텍처 개요

pgvector 기반 RAG 시스템의 전체 데이터 흐름은 다음과 같습니다:

1. 문서 준비: 원본 문서를 청킹(chunk)으로 분할합니다.

2. 임베딩 생성: Vertex AI Embedding API(text-embedding-005)로 각 청크를 벡터로 변환합니다.

3. DB 저장: PostgreSQL + pgvector 테이블에 텍스트와 벡터를 함께 저장합니다.

4. 벡터 검색: 사용자 쿼리를 임베딩하고, DB 에서 코사인 유사도 검색을 수행합니다.

5. 응답 생성: 검색된 컨텍스트를 Gemini 에 전달하여 응답을 생성합니다.

ChromaDB(로컬) vs pgvector(프로덕션) 비교

항목	ChromaDB	pgvector (PostgreSQL)
유형	전용 벡터 데이터베이스	PostgreSQL 확장
실행 환경	인메모리 / 로컬 파일	독립 서버 (Cloud SQL,

		Docker)
데이터 지속성	기본 임시적 (설정으로 영구화 가능)	완전한 영구 저장 + ACID 트랜잭션
확장성	제한적 (단일 노드)	높음 (Cloud SQL HA, AlloyDB 분산)
하이브리드 검색	메타데이터 필터링	SQL WHERE + 벡터 검색 통합
모니터링/백업	별도 구성 필요	PostgreSQL 표준 도구 활용
적합한 용도	프로토타입, 학습, PoC	프로덕션, 기존 DB 와 통합

로컬 환경 구성 (Docker)

GCP Cloud SQL 을 사용하지 않고 로컬에서 실습하려면 Docker 로 PostgreSQL + pgvector 환경을 구성합니다. 함께 제공되는 docker-compose.yml 을 사용하세요:

```
# docker-compose.yml 실행
docker-compose up -d

# 접속 확인
docker exec -it pgvector-db psql -U vectoruser -d vectordb

# pgvector 확장 확인
vectordb=# SELECT extversion FROM pg_extension WHERE extname = 'vector';
```

상세한 실습 코드는 06_db_pgvector.py 와 docker-compose.yml 을 참고하세요.

6.3 하이브리드 검색: SQL 필터 + 벡터 유사도

pgvector 의 가장 큰 장점 중 하나는 SQL 의 표현력과 벡터 검색을 결합할 수 있다는 점입니다. ChromaDB 에서는 메타데이터 필터링만 가능했지만, pgvector 에서는 복잡한 비즈니스 로직을 SQL 로 표현할 수 있습니다.

하이브리드 검색 예시

예를 들어 전자상거래 상품 DB 에서 "가격 5 만원 이하이면서 설명이 '무선 이어폰'과 유사한 상품"을 찾는 쿼리:

```
SELECT product_name, price, description,
       1 - (description_embedding <=> query_embedding) AS similarity
FROM products
WHERE price <= 50000              -- SQL 필터 (비즈니스 로직)
  AND category = '전자기기'       -- SQL 필터
  AND is_available = true         -- SQL 필터
ORDER BY description_embedding <=> query_embedding  -- 벡터 유사도
LIMIT 5;
```

활용 시나리오

시나리오	SQL 필터	벡터 검색
상품 추천	가격대, 카테고리, 재고 상태	상품 설명 유사도
사내 문서 검색	부서, 문서 유형, 작성일	문서 내용 유사도
고객 지원	고객 등급, 상품 카테고리	질문과 유사한 FAQ
채용 매칭	근무지, 경력, 연봉	이력서 내용과 직무 유사도

TIP: 하이브리드 검색에서는 SQL 필터로 먼저 후보를 줄인 후 벡터 검색을 수행하면 성능이 크게 향상됩니다. pgvector 의 인덱스는 WHERE 절 조건과 함께 사용될 수 있습니다.

6.4 BigQuery + Vertex AI 연동

BigQuery 는 GCP 의 서버리스 데이터 웨어하우스로, SQL 쿼리 내에서 직접 Vertex AI 의 Gemini 모델을 호출할 수 있습니다. 별도의 애플리케이션 코드 없이 SQL 만으로 대규모 데이터에 대한 AI 분석이 가능합니다.

AWS 비교: AWS 에서는 Amazon Redshift ML 이 SageMaker 모델을 SQL 에서 호출하는 것과 유사합니다. BigQuery ML.GENERATE_TEXT 는 Gemini 모델을 SQL 함수로 직접 호출하므로 더 간결합니다.

ML.GENERATE_TEXT: SQL 에서 Gemini 호출

ML.GENERATE_TEXT 함수를 사용하면 BigQuery 테이블의 데이터를 Gemini 에 전달하고 결과를 받을 수 있습니다:

```
-- BigQuery 에서 원격 모델 연결 생성
CREATE OR REPLACE MODEL `my_dataset.gemini_model`
REMOTE WITH CONNECTION `us-central1.my-connection`
OPTIONS (endpoint = 'gemini-2.0-flash');

-- SQL 에서 Gemini 호출
SELECT ml_generate_text_result, product_name
FROM ML.GENERATE_TEXT(
  MODEL `my_dataset.gemini_model`,
  (
    SELECT CONCAT('다음 상품의 마케팅 문구를 작성해줘: ',
          product_name, ' - ', description) AS prompt,
          product_name
    FROM `my_dataset.products`
    LIMIT 10
  ),
  STRUCT(0.4 AS temperature, 256 AS max_output_tokens)
);
```

ML.GENERATE_EMBEDDING: SQL 에서 임베딩 생성

BigQuery 에서 직접 텍스트 임베딩을 생성할 수도 있습니다:

```
-- 임베딩 모델 연결
CREATE OR REPLACE MODEL `my_dataset.embedding_model`
REMOTE WITH CONNECTION `us-central1.my-connection`
OPTIONS (endpoint = 'text-embedding-005');
```

```
-- 텍스트 임베딩 생성
SELECT * FROM ML.GENERATE_EMBEDDING(
  MODEL `my_dataset.embedding_model`,
  (SELECT description AS content FROM `my_dataset.products`)
);
```

BigQuery VECTOR_SEARCH

BigQuery 는 자체 벡터 검색 기능도 제공합니다. 임베딩이 저장된 테이블에서 유사한 항목을 검색할 수 있습니다:

```
SELECT query.query_text, base.product_name, base.description,
       distance
FROM VECTOR_SEARCH(
  TABLE `my_dataset.product_embeddings`,
  'embedding_column',
  (SELECT ml_generate_embedding_result AS embedding,
          content AS query_text
   FROM ML.GENERATE_EMBEDDING(
     MODEL `my_dataset.embedding_model`,
     (SELECT '무선 이어폰 추천' AS content))),
  top_k => 5
);
```

BigQuery + Vertex AI 활용 시나리오

시나리오	설명	사용 함수
대규모 텍스트 분석	수백만 건의 리뷰/피드백 감정 분석	ML.GENERATE_TEXT
자동 분류	상품/티켓 자동 카테고리 분류	ML.GENERATE_TEXT
유사 항목 검색	상품 유사도 검색, 중복 감지	ML.GENERATE_EMBEDDING + VECTOR_SEARCH

문서 요약	대규모 문서의 일괄 요약 생성	ML.GENERATE_TEXT
데이터 품질 검증	비정형 텍스트 필드의 품질 평가	ML.GENERATE_TEXT

TIP: BigQuery ML 은 대규모 데이터에 AI 를 적용할 때 가장 효율적입니다. 애플리케이션 코드 없이 SQL 만으로 데이터 분석과 AI 통합이 가능하므로, 데이터 분석가에게 특히 유용합니다.

6.5 실습 파일 가이드

이번 Chapter 의 실습 파일 구성은 다음과 같습니다:

파일	설명	사전 요구사항
docker-compose.yml	PostgreSQL 16 + pgvector 로컬 환경	Docker 설치
06_db_pgvector.py	pgvector 기반 RAG 전체 실습 (6 개 실습)	Docker 환경 + Vertex AI 인증
07_bigquery_vertexai.py	BigQuery + Vertex AI 연동 실습 (3 개 실습)	GCP 프로젝트 + BigQuery API

06_db_pgvector.py 실습 목록

실습	제목	학습 목표
실습 A	Docker 로 pgvector 환경 구성	PostgreSQL + pgvector 설치 및 확인
실습 B	벡터 테이블 생성 및 데이터 삽입	Vertex AI Embedding 으로 벡터화 + DB 저장
실습 C	벡터 유사도 검색	코사인 거리 기반 벡터 검색
실습 D	하이브리드 검색	SQL WHERE + 벡터 검색 결합
실습 E	LangChain PGVector RAG 체인	LangChain 통합 RAG 파이프라인
실습 F	Cloud SQL 연결 (참고)	GCP 운영 환경 연결 방법

07_bigquery_vertexai.py 실습 목록

실습	제목	학습 목표
실습 A	BigQuery 테이블 생성 및 조회	BigQuery Python 클라이언트 기본 사용법
실습 B	ML.GENERATE_TEXT 로 Gemini	SQL 에서 직접 LLM 호출

	호출	
실습 C	ML.GENERATE_EMBEDDING 으로 임베딩	SQL 에서 직접 임베딩 생성

각 실습 파일은 독립적으로 실행 가능하며, 파일 상단의 안내에 따라 환경을 설정한 후 각 실습을 순서대로 진행하세요.

7. Vertex AI Agent Builder

지금까지 프롬프트 엔지니어링, RAG, 데이터베이스 연동 등을 통해 LLM 기반 애플리케이션의 핵심 요소를 학습했습니다. 이번 Chapter 에서는 이러한 기술들을 하나로 통합하여 자율적으로 작업을 수행하는 AI 에이전트(Agent)를 구축하는 플랫폼인 Vertex AI Agent Builder 를 소개합니다.

> **AWS 비교:** AWS 에서는 Amazon Bedrock Agents 가 유사한 역할을 합니다. Agent Builder 는 Google 의 Gemini 모델 + Google Search Grounding + Vertex AI Search 등 Google 생태계와 긴밀하게 통합되어 있으며, Agent Development Kit(ADK)라는 오픈소스 프레임워크를 함께 제공합니다.

7.0 AI 에이전트란?

AI 에이전트는 단순한 챗봇과 달리 사용자의 목표를 이해하고 외부 도구를 활용하여 자율적으로 작업을 수행하는 시스템입니다. 일반 챗봇은 사용자 입력에 대해 텍스트 응답만 생성하지만, 에이전트는 상황을 판단하고 적절한 도구(Tool)를 선택하여 실제 작업을 실행합니다.

챗봇 vs 에이전트 비교

항목	일반 챗봇	AI 에이전트
동작 방식	질문 → 텍스트 응답	목표 → 계획 → 도구 실행 → 결과
외부 연동	없음 (텍스트만 생성)	API, DB, 파일시스템, 웹 검색 등
자율성	사용자 지시마다 1 회 응답	멀티스텝 추론으로 자율 수행
상태 관리	대화 히스토리만 유지	작업 상태 + 컨텍스트 지속 추적
예시	"오늘 날씨 어때?" → 텍스트 응답	"내일 서울 출장 일정 잡아줘" → 캘린더 확인 → 항공편 검색 → 예약 실행

에이전트의 핵심 구성 요소

Foundation Model (기반 모델): 에이전트의 두뇌 역할을 합니다. Gemini 2.0 Flash/Pro 등이 추론, 계획 수립, 도구 선택을 담당합니다.

Tools (도구): 에이전트가 외부와 상호작용하는 수단입니다. API 호출, 데이터베이스 쿼리, Google 검색, 코드 실행 등이 포함됩니다.

Orchestration (오케스트레이션): 모델이 어떤 도구를 언제, 어떤 순서로 호출할지 결정하는 추론 루프입니다. ReAct(Reasoning + Acting) 패턴이 대표적입니다.

Memory (메모리): 단기 메모리(현재 대화)와 장기 메모리(세션 간 정보)를 관리하여 에이전트가 문맥을 유지할 수 있게 합니다.

> **TIP:** 에이전트는 "LLM + 도구 사용 + 자율 추론 루프"를 결합한 것입니다. LLM 이 단순히 텍스트를 생성하는 것을 넘어, 실제 행동(Action)을 취할 수 있게 됩니다.

에이전트 동작 흐름 (ReAct 패턴)

대부분의 AI 에이전트는 ReAct(Reasoning + Acting) 패턴으로 동작합니다. 이 패턴에서 모델은 다음 루프를 반복합니다:

1. Thought (추론): 현재 상황을 분석하고 다음 행동을 계획합니다.

2. Action (실행): 적절한 도구를 선택하고 입력값을 결정하여 호출합니다.

3. Observation (관찰): 도구 실행 결과를 확인하고 목표 달성 여부를 판단합니다.

4. 반복/완료: 목표가 달성되지 않았으면 1 번으로 돌아가 추가 행동을 수행하고, 완료되면 최종 답변을 생성합니다.

예시: 사용자가 "서울 날씨 확인하고, 비가 오면 내일 미팅을 실내로 변경해줘"라고 요청하면:

```
Thought: 먼저 서울 날씨를 확인해야 함
Action: weather_api.get(location="서울", date="내일")
Observation: 내일 서울 비 예보 (강수확률 80%)

Thought: 비가 오므로 미팅 장소를 실내로 변경해야 함
Action: calendar_api.update(meeting_id="abc", location="실내 회의실 A")
Observation: 미팅 장소 변경 완료

Final Answer: 내일 서울에 비가 예보되어 있어서 미팅 장소를
  실내 회의실 A 로 변경했습니다.
```

7.1 Vertex AI Agent Builder 핵심 기능

Vertex AI Agent Builder 는 Google Cloud 에서 AI 에이전트를 설계, 개발, 배포하기 위한 통합 플랫폼입니다. 노코드(No-code) 콘솔 UI 부터 코드 기반 프레임워크(ADK)까지 다양한 수준의 개발 방식을 지원합니다.

Agent Builder 아키텍처 구성

Agent Builder 플랫폼은 크게 세 가지 핵심 컴포넌트로 구성됩니다:

컴포넌트	설명	대상 사용자
Agent Builder Console	Google Cloud 콘솔에서 노코드/로우코드로 에이전트를 설계하고 테스트하는 UI 도구	비즈니스 사용자, 빠른 프로토타이핑
Agent Development Kit (ADK)	Python 기반 오픈소스 에이전트	개발자, 커스텀 에이전트 구축

	개발 프레임워크. 코드로 에이전트를 정의하고 로컬에서 테스트 가능	
Agent Engine	에이전트를 프로덕션 환경에 배포하고 관리하는 관리형 런타임. 세션 관리, 로깅, 모니터링 포함	운영팀, 프로덕션 배포

TIP: Agent Builder Console 로 빠르게 프로토타입을 만들고, ADK 로 정교한 커스텀 로직을 구현한 뒤, Agent Engine 에 배포하는 것이 권장 워크플로우입니다.

노코드 에이전트 생성 (Console UI)

Agent Builder Console 에서는 코드 없이 에이전트를 생성할 수 있습니다. Google Cloud Console 에서 다음 경로로 접근합니다:

```
Google Cloud Console
  → Vertex AI
    → Agent Builder
      → + Create Agent
```

에이전트 생성 시 설정하는 주요 항목:

설정 항목	설명	예시
Agent Name	에이전트 이름	고객 지원 에이전트
Goal	에이전트의 목표/역할을 자연어로 기술	"고객의 주문 상태를 조회하고 반품/교환을 처리합니다"
Instructions	에이전트가 따라야 할 행동 규칙	"항상 주문번호를 먼저 확인하세요"
Model	사용할 Gemini 모델 선택	Gemini 2.0 Flash (빠른 응답)
Tools	에이전트가 사용할 도구 연결	주문 조회 API, 반품 처리 API

도구(Tool) 연결

에이전트의 핵심은 외부 시스템과 상호작용할 수 있는 도구(Tool)입니다. Agent Builder 에서 연결 가능한 도구 유형은 다음과 같습니다:

도구 유형	설명	활용 예시
OpenAPI Tool	OpenAPI(Swagger) 스펙으로 정의된 REST API 를 에이전트가 호출	사내 주문 관리 API, 재고 확인 API
Data Store Tool	Vertex AI Search 데이터스토어에서 정보를 검색 (RAG)	사내 문서 검색, FAQ 데이터베이스
Google Search	Google Search Grounding 을 통해 최신 웹 정보 검색	최신 뉴스, 실시간 정보 질의
Code Interpreter	에이전트가 Python 코드를 생성하고 실행 (데이터 분석, 계산)	데이터 분석, 차트 생성, 수학 계산
Extensions	Google Workspace(Gmail, Calendar, Drive 등), BigQuery 등 사전 구축된 연결	이메일 전송, 일정 확인, 스프레드시트 조회

OpenAPI Tool 예시: 주문 조회 API 를 OpenAPI 스펙(YAML/JSON)으로 작성하여 에이전트에 연결하면, 에이전트가 자연어 요청을 해석하여 적절한 API 엔드포인트를 호출합니다.

```yaml
# OpenAPI 스펙 예시 (YAML)
openapi: 3.0.0
info:
  title: Order Management API
  version: 1.0.0
paths:
  /orders/{order_id}:
    get:
      summary: 주문 상태 조회
      operationId: getOrderStatus
      parameters:
        - name: order_id
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: 주문 정보 반환
```

TIP: Data Store Tool 은 Chapter 3 에서 학습한 Grounding 및 Chapter 4 의 RAG 개념과 직접 연결됩니다. 에이전트가 기업 내부 문서를 검색하여 답변하는 가장 쉬운 방법입니다.

대화 흐름 설계 및 테스트

Agent Builder Console 에서는 에이전트의 동작을 시뮬레이션하고 테스트할 수 있는 통합 환경을 제공합니다:

Simulator (시뮬레이터): 에이전트와 실시간 대화를 테스트합니다. 에이전트가 어떤 도구를 호출하는지, 추론 과정은 어떤지 단계별로 확인할 수 있습니다.

Trace View: 에이전트의 ReAct 추론 과정을 시각적으로 보여줍니다. Thought → Action → Observation 각 단계를 확인하여 디버깅할 수 있습니다.

Test Cases: 사전 정의된 테스트 시나리오를 작성하고 자동으로 에이전트를 평가할 수 있습니다. 예상 응답과 실제 응답을 비교하여 품질을 관리합니다.

에이전트 배포 옵션

테스트가 완료된 에이전트는 다양한 채널로 배포할 수 있습니다:

배포 채널	설명
Dialogflow CX Messenger	웹사이트에 임베드할 수 있는 채팅 위젯
REST API	커스텀 애플리케이션에서 API 로 에이전트 호출
Agent Engine	관리형 런타임에 배포하여 세션 관리, 로깅, 스케일링 자동화
Google Chat / Slack 등	메신저 플랫폼 통합 (Webhook 또는 확장 사용)

7.2 Agent Development Kit (ADK)

Agent Development Kit(ADK)는 Google 이 공개한 오픈소스 Python 프레임워크로, 코드 기반으로 AI 에이전트를 개발하고 테스트할 수 있습니다. Agent Builder Console 의 노코드 방식보다 더 정교한 제어가 필요할 때 사용합니다.

ADK 주요 특징

오픈소스 Python 프레임워크: pip install google-adk 로 설치 가능하며 GitHub 에서 소스 코드가 공개되어 있습니다.

멀티 에이전트 아키텍처: 여러 에이전트가 협력하는 멀티 에이전트 시스템을 계층적으로 구성할 수 있습니다.

다양한 모델 지원: Gemini 뿐 아니라 LiteLLM 을 통해 다른 LLM 모델도 사용할 수 있습니다.

풍부한 도구 생태계: 함수를 도구로 바로 사용하거나, MCP(Model Context Protocol) 서버, API Hub, LangChain 도구 등과 연동할 수 있습니다.

로컬 개발 환경: 로컬에서 에이전트를 개발하고 내장 UI(adk web)로 테스트한 후, Agent Engine 에 배포합니다.

ADK 에이전트 코드 구조

ADK 로 에이전트를 만들 때의 기본 프로젝트 구조와 핵심 코드 패턴을 소개합니다.

프로젝트 구조:

```
my_agent/
  __init__.py          # 에이전트 패키지 초기화
  agent.py             # 에이전트 정의 (핵심 파일)
  tools.py             # 커스텀 도구 정의 (선택)
  .env                 # 환경변수 (GOOGLE_CLOUD_PROJECT 등)
```

핵심 코드 패턴 (agent.py):

```
from google.adk.agents import Agent
from google.adk.tools import google_search

# 커스텀 도구를 일반 Python 함수로 정의
def get_weather(city: str) -> dict:
    """지정된 도시의 현재 날씨를 조회합니다."""
    # 실제로는 날씨 API 를 호출
    return {"city": city, "temp": "18°C", "condition": "맑음"}

# 에이전트 정의
root_agent = Agent(
    name="weather_assistant",
    model="gemini-2.0-flash",
    description="날씨 정보를 제공하는 에이전트",
    instruction="""
        당신은 날씨 정보를 제공하는 도우미입니다.
        사용자가 날씨를 물어보면 get_weather 도구를 사용하세요.
```

```
        항상 한국어로 친절하게 답변하세요.
    """,
    tools=[get_weather, google_search],  # 도구 목록
)
```

TIP: ADK 의 핵심 장점은 일반 Python 함수를 도구로 사용할 수 있다는 것입니다. 함수의 이름, 타입 힌트, docstring 을 기반으로 LLM 이 자동으로 도구 사용법을 이해합니다.

멀티 에이전트 구성

ADK 는 여러 에이전트가 역할을 분담하여 협력하는 멀티 에이전트 시스템을 지원합니다. 루트 에이전트가 전체를 조율하고, 서브 에이전트들이 전문 작업을 수행합니다.

```
from google.adk.agents import Agent

# 서브 에이전트 1: 날씨 전문
weather_agent = Agent(
    name="weather_agent",
    model="gemini-2.0-flash",
    description="날씨 정보를 조회하고 분석합니다",
    tools=[get_weather, get_forecast],
)

# 서브 에이전트 2: 일정 관리 전문
calendar_agent = Agent(
    name="calendar_agent",
    model="gemini-2.0-flash",
    description="캘린더 일정을 조회하고 관리합니다",
    tools=[get_events, create_event, update_event],
)

# 루트 에이전트: 요청을 분석하고 적절한 서브 에이전트에 위임
root_agent = Agent(
    name="personal_assistant",
    model="gemini-2.0-flash",
    description="개인 비서 에이전트",
    instruction="""
        사용자 요청을 분석하여 적절한 전문 에이전트에게 위임하세요.
        날씨 관련 → weather_agent
        일정 관련 → calendar_agent
    """,
    sub_agents=[weather_agent, calendar_agent],  # 서브 에이전트
)
```

ADK 로컬 테스트 및 Agent Engine 배포

ADK 는 로컬 개발 환경에서 에이전트를 테스트하고, 준비되면 Agent Engine 에 배포하는 워크플로우를 지원합니다.

로컬 테스트:

```
# ADK 설치
pip install google-adk

# 내장 웹 UI 로 에이전트 테스트 (브라우저에서 대화 가능)
adk web my_agent/

# CLI 로 에이전트 테스트
adk run my_agent/
```

Agent Engine 배포:

```
from google.adk.agents import Agent
from vertexai import agent_engines

# 에이전트를 Agent Engine 에 배포
remote_agent = agent_engines.create(
    agent_engine=root_agent,
    display_name="my-production-agent",
    description="프로덕션 개인 비서 에이전트",
)

# 배포된 에이전트에 쿼리
response = remote_agent.query(
    input="내일 서울 날씨 어때?",
    user_id="user-123",
)
print(response)
```

Agent Engine 에 배포하면 다음 기능이 자동으로 제공됩니다:

세션 관리: 사용자별 대화 세션을 자동으로 관리하고 지속합니다.

스케일링: 트래픽에 따라 자동으로 확장/축소됩니다.

로깅/모니터링: Cloud Logging/Monitoring 과 통합되어 에이전트 동작을 추적할 수 있습니다.

보안: IAM 기반 접근 제어와 VPC Service Controls 를 지원합니다.

7.3 AWS Bedrock Agents 와의 비교

AWS 경험이 있는 개발자를 위해 Vertex AI Agent Builder 와 Amazon Bedrock Agents 의 주요 기능을 비교합니다.

플랫폼 기능 대응 비교

기능	Vertex AI Agent Builder	Amazon Bedrock Agents
에이전트 정의	Goal + Instructions (자연어)	Agent Instructions (자연어)

기반 모델	Gemini 2.0 Flash/Pro, Gemini 2.5	Claude, Nova, Llama, Titan 등
도구 연결	OpenAPI Tool, Extensions	Action Groups (Lambda 기반)
데이터 검색 (RAG)	Data Store Tool (Vertex AI Search)	Knowledge Bases (OpenSearch, S3 등)
웹 검색	Google Search Grounding (내장)	별도 구성 필요
코드 실행	Code Interpreter (내장)	Code Interpreter (내장)
개발 프레임워크	ADK (오픈소스 Python)	Agents SDK / LangChain 등
멀티 에이전트	ADK 멀티 에이전트 + Agent Garden	Multi-agent collaboration
배포 런타임	Agent Engine (관리형)	Bedrock Agent (관리형)
테스트 도구	Simulator + Trace View	Test Window (콘솔)
노코드 생성	Console UI 지원	Console UI 지원

핵심 개념 매핑

GCP 개념	AWS 대응	설명
Agent Builder Console	Bedrock Agents 콘솔	노코드 에이전트 생성 UI
Tools (OpenAPI)	Action Groups (Lambda)	에이전트가 호출하는 외부 기능. GCP 는 OpenAPI 스펙을 직접 사용하고, AWS 는 Lambda 함수로 래핑
Data Store Tool	Knowledge Bases	RAG 기반 문서 검색. GCP 는 Vertex AI Search, AWS 는 OpenSearch/S3 활용
Agent Engine	Bedrock Agent 런타임	프로덕션 배포 및 세션 관리
ADK (Agent Dev Kit)	Agents SDK / 서드파티	코드 기반 에이전트 개발 프레임워크
Google Search Grounding	웹 검색 (별도 구성)	실시간 웹 정보 연동. GCP 는 내장 지원

AWS 비교: 가장 큰 차이점은 도구 연결 방식입니다. AWS Bedrock Agents 는 Action Groups 에서 Lambda 함수를 호출하는 구조이므로, 모든 외부 API 를 Lambda 로 래핑해야 합니다. 반면 GCP Agent Builder 는 OpenAPI 스펙을 직접 사용할 수 있어 기존 REST API 를 그대로 연결할 수 있습니다. 또한 Google Search Grounding 이 내장되어 있어 최신 정보 검색이 간편합니다.

7.4 활용 사례 및 설계 가이드

Agent Builder 를 활용할 수 있는 대표적인 업무 시나리오와 설계 시 고려사항을 정리합니다.

대표 활용 사례

시나리오	에이전트 구성	사용 도구
고객 지원 자동화	고객 문의 → FAQ 검색 → 주문 조회 → 반품 처리	Data Store (FAQ), OpenAPI (주문/반품 API)
사내 IT 헬프데스크	직원 질문 → 사내 문서 검색 → 티켓 생성	Data Store (IT 매뉴얼), Extensions (JIRA API)
영업 지원 에이전트	고객사 정보 조회 → CRM 업데이트 → 미팅 일정 설정	OpenAPI (CRM), Extensions (Calendar)
데이터 분석 비서	자연어 질의 → SQL 생성/실행 → 차트 생성	Code Interpreter, OpenAPI (BigQuery)

여행 계획 에이전트	선호도 파악 → 항공편 검색 → 호텔 추천 → 일정 생성	Google Search, OpenAPI (예약 API)

에이전트 설계 시 고려사항

명확한 Goal 정의: 에이전트의 역할과 범위를 명확히 정의하세요. 너무 범용적인 에이전트보다 특정 업무에 집중된 에이전트가 성능이 좋습니다.

적절한 도구 수: 에이전트에 연결하는 도구는 5~10 개 이내가 권장됩니다. 도구가 너무 많으면 모델이 올바른 도구를 선택하는 데 어려움을 겪습니다.

도구 설명 품질: 각 도구의 이름, 설명, 파라미터 설명이 명확해야 합니다. LLM 이 이 정보를 기반으로 도구 사용 여부를 판단합니다.

에러 처리: 도구 호출 실패 시 에이전트가 어떻게 대응할지 Instructions 에 명시하세요. 예: "API 호출이 실패하면 사용자에게 안내하고 대안을 제시하세요."

보안 고려: 에이전트가 접근하는 데이터와 API 의 권한을 최소화하세요. 서비스 계정에 필요한 최소 권한만 부여합니다.

> **TIP:** 에이전트를 처음 도입할 때는 FAQ 검색 + 간단한 API 1~2 개 조합의 소규모 에이전트로 시작하여 점진적으로 기능을 확장하는 것을 권장합니다.

7.5 요약 및 다음 단계

이번 Chapter 에서 학습한 핵심 내용을 정리합니다:

AI 에이전트: LLM + 도구(Tool) + 자율 추론 루프를 결합하여 실제 작업을 수행하는 시스템입니다.

Agent Builder: Console(노코드) + ADK(코드) + Agent Engine(배포)의 세 가지 컴포넌트로 구성된 통합 플랫폼입니다.

ADK: Python 함수를 도구로 사용하고, 멀티 에이전트 시스템을 구성할 수 있는 오픈소스 프레임워크입니다.

AWS 대응: Amazon Bedrock Agents 가 유사한 기능을 제공하며, Action Groups(Lambda)와 Knowledge Bases 가 각각 Tools 와 Data Store 에 대응됩니다.

에이전트 기술은 빠르게 발전하고 있으며, Google Cloud 의 최신 문서를 참고하여 새로운 기능과 모범 사례를 확인하시기 바랍니다.

참고 리소스

Vertex AI Agent Builder 공식 문서: https://cloud.google.com/products/agent-builder

Agent Development Kit (ADK) GitHub: https://github.com/google/adk-python

Google Cloud 에이전트 가이드: https://cloud.google.com/vertex-ai/generative-ai/docs/agent-builder/overview

부록: GCP 환경 준비 상세 가이드

부록 0-1. GCP 계정 생성 및 무료 체험

Google Cloud Platform(GCP)을 사용하려면 Google 계정과 GCP 프로젝트가 필요합니다. GCP 는 신규 사용자에게 $300 크레딧(약 40 만원)과 90 일의 무료 체험 기간을 제공합니다.

무료 체험 프로그램 개요

항목	내용
크레딧	$300 (약 40 만원) 무료 크레딧
기간	가입일로부터 90 일
결제 수단	신용카드/체크카드 등록 필요 (자동 결제되지 않음)
자동 전환	무료 체험 종료 후 수동으로 유료 전환하지 않으면 서비스 중지
사용 가능 서비스	Vertex AI, Cloud SQL, BigQuery 등 대부분의 GCP 서비스
제한 사항	GPU VM 일부 제한, 리소스 할당량 제한

GCP 계정 생성 절차

Step 1. Google 계정이 없다면 https://accounts.google.com 에서 계정을 생성합니다.

Step 2. https://cloud.google.com 에 접속하여 "무료로 시작하기" 버튼을 클릭합니다.

Step 3. 국가/지역을 선택하고 서비스 약관에 동의합니다.

Step 4. 결제 프로필을 생성합니다:

```
계정 유형: 개인 또는 비즈니스 선택
이름/주소: 결제 정보 입력
결제 수단: 신용카드 또는 체크카드 번호 입력

※ 카드는 본인 확인용이며, 무료 체험 기간에는 과금되지 않습니다.
※ 무료 체험 종료 후에도 수동으로 업그레이드하지 않으면 과금되지 않습니다.
```

Step 5. 계정 생성이 완료되면 자동으로 Google Cloud Console(https://console.cloud.google.com)로 이동합니다.

Step 6. 상단에 "무료 체험 활성화됨" 메시지와 잔여 크레딧이 표시되는지 확인합니다.

결제 알림 설정

예상치 못한 비용이 발생하지 않도록 결제 알림을 설정하는 것이 중요합니다:

Step 1. Cloud Console → 결제(Billing) → 예산 및 알림(Budgets & alerts) 이동

Step 2. "예산 만들기(Create budget)" 클릭

Step 3. 예산 이름 입력 (예: "Vertex AI 교육 예산")

Step 4. 대상 금액 설정 (예: $50)

Step 5. 알림 임계값 설정: 50%, 80%, 100%에서 이메일 알림

> **TIP:** 무료 체험 기간에도 예산 알림을 설정해두면 크레딧 사용량을 모니터링할 수 있습니다. $50 예산에 50% 알림을 설정하면 $25 사용 시 알림을 받습니다.

부록 0-2. 프로젝트 생성 및 API 활성화

GCP 의 모든 리소스는 프로젝트 단위로 관리됩니다. Vertex AI 를 사용하려면 프로젝트를 생성하고 필요한 API 를 활성화해야 합니다.

프로젝트 생성

Step 1. Cloud Console(https://console.cloud.google.com) 접속

Step 2. 상단 프로젝트 선택 드롭다운 클릭 → "새 프로젝트(New Project)" 클릭

Step 3. 프로젝트 설정:

```
프로젝트 이름: vertexai-training (원하는 이름으로 변경 가능)
조직: 개인 계정이면 "조직 없음" 선택
위치: 기본값 유지

→ "만들기(Create)" 클릭
```

Step 4. 프로젝트가 생성되면 상단에서 해당 프로젝트가 선택되었는지 확인합니다.

Step 5. 프로젝트 ID 를 기록해둡니다 (이후 SDK 설정에서 사용):

```
# 프로젝트 ID 확인 위치
Cloud Console → 프로젝트 설정(Project settings) → 프로젝트 ID

# 예: vertexai-training-123456
# 프로젝트 이름과 ID 는 다를 수 있으므로 ID 를 사용해야 합니다.
```

API 활성화

Vertex AI 와 관련 서비스를 사용하려면 해당 API 를 활성화해야 합니다. Cloud Console → API 및 서비스(APIs & Services) → 라이브러리(Library)에서 각 API 를 검색하고 "사용(Enable)" 버튼을 클릭합니다.

필수 API 목록:

API 이름	용도	필수 여부
Vertex AI API	Gemini 모델 호출, 임베딩 생성, RAG Engine 등 Vertex AI 핵심 기능	필수
Cloud Storage API	GCS 버킷에 파일 업로드/다운로드 (RAG 데이터, Cloud Functions 배포)	필수
Cloud Functions API	서버리스 함수 배포 및 실행 (Chapter 5)	필수
Cloud Build API	Cloud Functions 배포 시 빌드 프로세스에 사용	필수
Cloud Run API	Cloud Functions 2 세대 실행 환경	필수
BigQuery API	BigQuery 테이블 조회/생성, ML 함수 사용 (Chapter 6)	Chapter 6
Cloud SQL Admin API	Cloud SQL 인스턴스 관리 (Chapter 6, GCP 환경)	Chapter 6
Cloud Resource Manager API	프로젝트 리소스 관리	권장

gcloud CLI 로 한 번에 활성화하기:

```
# 필수 API 일괄 활성화
gcloud services enable \
  aiplatform.googleapis.com \
  storage.googleapis.com \
  cloudfunctions.googleapis.com \
  cloudbuild.googleapis.com \
  run.googleapis.com \
  bigquery.googleapis.com \
  sqladmin.googleapis.com \
  cloudresourcemanager.googleapis.com
```

TIP: API 활성화는 한 번만 하면 되며, 활성화 자체로는 비용이 발생하지 않습니다. 실제 API 를 호출할 때만 과금됩니다.

부록 0-3. IAM 및 서비스 계정

GCP 의 IAM(Identity and Access Management)은 누가(주체) 어떤 리소스에 어떤 작업(역할)을 수행할 수 있는지 제어합니다. 로컬 개발 환경에서 Vertex AI 를 호출하려면 적절한 권한이 설정된 인증 정보가 필요합니다.

인증 방법 비교

인증 방법	설명	적합한 환경
사용자 인증 (ADC)	gcloud auth application-default login 으로 로컬 사용자 인증	로컬 개발/학습 환경
서비스 계정 키	JSON 키 파일을 생성하여 환경변수로 설정	CI/CD, 서버 환경
Workload Identity	GKE, Cloud Run 등에서 자동 인증	GCP 내부 서비스

방법 1: 사용자 인증 (ADC) — 학습 환경 권장

Application Default Credentials(ADC)는 가장 간단한 인증 방법입니다. 로컬 개발과 학습 환경에 적합합니다:

```
# 1. GCP 사용자 인증 (브라우저에서 로그인)
gcloud auth login

# 2. Application Default Credentials 설정
gcloud auth application-default login

# 3. 프로젝트 설정
gcloud config set project YOUR_PROJECT_ID

# 4. 인증 확인
gcloud auth list
gcloud config get-value project
```

TIP: ADC 를 사용하면 별도의 키 파일 없이 Python SDK 에서 자동으로 인증됩니다. 교육 실습에서는 이 방법을 권장합니다.

방법 2: 서비스 계정 — 프로덕션 환경

서비스 계정은 사람이 아닌 애플리케이션이 GCP 리소스에 접근할 때 사용하는 특수 계정입니다. 프로덕션 환경에서는 서비스 계정을 사용합니다.

Step 1. 서비스 계정 생성:

```
# Cloud Console → IAM → 서비스 계정 → 만들기
# 또는 gcloud CLI:

gcloud iam service-accounts create vertexai-sa \
  --display-name="Vertex AI Service Account"
```

Step 2. 역할(Role) 부여:

```
PROJECT_ID=$(gcloud config get-value project)
SA_EMAIL=vertexai-sa@${PROJECT_ID}.iam.gserviceaccount.com

# Vertex AI 사용자 역할
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SA_EMAIL" \
  --role="roles/aiplatform.user"
```

```
# Cloud Storage 관리자 (RAG 데이터 업로드용)
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SA_EMAIL" \
  --role="roles/storage.admin"
```

Step 3. 키 파일 생성 및 환경변수 설정:

```
# JSON 키 파일 다운로드
gcloud iam service-accounts keys create ~/vertexai-sa-key.json \
  --iam-account=$SA_EMAIL

# 환경변수 설정 (Linux/Mac)
export GOOGLE_APPLICATION_CREDENTIALS=~/vertexai-sa-key.json

# 환경변수 설정 (Windows PowerShell)
$env:GOOGLE_APPLICATION_CREDENTIALS="C:\Users\사용자\vertexai-sa-key.json"
```

주의: 서비스 계정 키 파일(JSON)은 비밀번호와 같은 민감 정보입니다. Git 저장소에 커밋하지 마세요. .gitignore 에 *.json 키 파일을 추가하세요.

주요 IAM 역할 정리

역할 (Role)	역할 ID	권한 범위
Vertex AI 사용자	roles/aiplatform.user	Gemini 호출, 임베딩 생성, 예측 요청
Vertex AI 관리자	roles/aiplatform.admin	모델 배포, 엔드포인트 관리, 파이프라인 실행
Storage 관리자	roles/storage.admin	GCS 버킷/객체 전체 관리
Storage 객체 뷰어	roles/storage.objectViewer	GCS 객체 읽기 전용
BigQuery 데이터 편집자	roles/bigquery.dataEditor	BigQuery 테이블 생성/수정/조회
Cloud SQL 클라이언트	roles/cloudsql.client	Cloud SQL 인스턴스 연결
Cloud Functions 개발자	roles/cloudfunctions.developer	함수 배포 및 관리

TIP: 최소 권한 원칙(Principle of Least Privilege)을 적용하세요. 학습 환경에서는 roles/aiplatform.user 와 roles/storage.admin 이면 대부분의 실습이 가능합니다.

부록 0-4. Cloud SDK(gcloud CLI) 설치

Google Cloud SDK(gcloud CLI)는 GCP 리소스를 명령줄에서 관리하는 도구입니다. 프로젝트 설정, API 활성화, 인증, 서비스 배포 등 다양한 작업에 사용됩니다.

운영체제별 설치 방법

Windows:

```
# 방법 1: 설치 프로그램 (권장)
# https://cloud.google.com/sdk/docs/install 에서
# GoogleCloudSDKInstaller.exe 다운로드 → 실행

# 방법 2: PowerShell
(New-Object Net.WebClient).DownloadFile(
  "https://dl.google.com/dl/cloudsdk/channels/rapid/GoogleCloudSDKInstaller.exe",
  "$env:TEMP\GoogleCloudSDKInstaller.exe")
& $env:TEMP\GoogleCloudSDKInstaller.exe
```

macOS:

```
# 방법 1: Homebrew (권장)
brew install --cask google-cloud-sdk

# 방법 2: 공식 스크립트
curl https://sdk.cloud.google.com | bash
exec -l $SHELL  # 셸 재시작
```

Linux (Debian/Ubuntu):

```
# APT 저장소 추가 및 설치
sudo apt-get install apt-transport-https ca-certificates gnupg curl
curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg
echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] \
  https://packages.cloud.google.com/apt cloud-sdk main" | \
  sudo tee /etc/apt/sources.list.d/google-cloud-sdk.list
sudo apt-get update && sudo apt-get install google-cloud-cli
```

초기 설정 (gcloud init)

SDK 설치 후 다음 명령으로 초기 설정을 진행합니다:

```
# 1. 초기화 (대화형 설정)
gcloud init

# 질문에 답변:
# - Google 계정 로그인 (브라우저 열림)
# - 프로젝트 선택 또는 생성
# - 기본 리전 설정 (asia-northeast3 = 서울 권장)

# 2. Application Default Credentials 설정
gcloud auth application-default login

# 3. 설정 확인
gcloud config list
```

자주 사용하는 gcloud 명령어

명령어	설명
gcloud config set project PROJECT_ID	기본 프로젝트 설정
gcloud config set compute/region asia-northeast3	기본 리전을 서울로 설정
gcloud auth list	인증된 계정 목록 확인
gcloud services list --enabled	활성화된 API 목록 확인
gcloud services enable aiplatform.googleapis.com	특정 API 활성화
gcloud projects describe PROJECT_ID	프로젝트 정보 조회
gcloud ai models list --region=us-central1	사용 가능한 AI 모델 목록
gcloud functions deploy FUNC --gen2	Cloud Functions 배포
gcloud sql instances list	Cloud SQL 인스턴스 목록
gcloud config configurations list	설정 프로필 목록

TIP: 여러 프로젝트를 사용하는 경우 gcloud config configurations create/activate 로 설정 프로필을 분리할 수 있습니다.

부록 0-5. VS Code + Cloud Code 설정

Visual Studio Code(VS Code)에 Cloud Code 확장을 설치하면 GCP 서비스와 통합된 개발 환경을 구성할 수 있습니다.

VS Code 설치

https://code.visualstudio.com 에서 운영체제에 맞는 버전을 다운로드하여 설치합니다.

Cloud Code 확장 설치

Step 1. VS Code 실행 → 왼쪽 사이드바의 확장(Extensions) 아이콘 클릭 (Ctrl+Shift+X)

Step 2. 검색창에 "Cloud Code" 입력

Step 3. Google Cloud Code 확장을 찾아 "Install" 클릭

Step 4. 설치 완료 후 VS Code 하단 상태 바에 "Cloud Code" 아이콘이 표시됩니다.

Cloud Code 주요 기능

기능	설명
GCP 프로젝트 연결	VS Code 에서 직접 GCP 프로젝트를 선택하고 전환
API Explorer	GCP API 를 VS Code 에서 탐색하고 테스트
Cloud Functions 배포	함수를 VS Code 에서 직접 배포 및 로그 확인
Cloud Run 배포	컨테이너 서비스를 VS Code 에서 배포
Secret Manager	시크릿을 VS Code 에서 관리
로그 뷰어	Cloud Logging 로그를 VS Code 에서 실시간 확인

Python 개발 환경 설정

Vertex AI 실습을 위한 Python 환경을 구성합니다:

Step 1. Python 3.9+ 설치 확인:

```
python --version  # 또는 python3 --version
```

Step 2. 가상환경 생성 및 활성화:

```
# 가상환경 생성
python -m venv venv

# 활성화 (Windows)
venv\Scripts\activate

# 활성화 (macOS/Linux)
source venv/bin/activate
```

Step 3. requirements.txt 로 패키지 설치:

```
# requirements.txt 내용
google-cloud-aiplatform>=1.60.0
langchain>=0.2.0
langchain-google-vertexai>=1.0.0
langchain-google-community>=1.0.0
langchain-community>=0.2.0
chromadb>=0.5.0
psycopg2-binary>=2.9.0
langchain-postgres>=0.0.6
google-cloud-bigquery>=3.20.0
python-dotenv>=1.0.0

# 설치 실행
pip install -r requirements.txt
```

Step 4. 환경변수 설정 (.env 파일):

```
# .env 파일 생성 (프로젝트 루트)
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_REGION=us-central1

# Python 에서 로드
from dotenv import load_dotenv
load_dotenv()
```

TIP: VS Code 에서 Python 확장과 Pylance 확장도 함께 설치하면 코드 자동완성, 타입 체크 등 개발 편의성이 크게 향상됩니다.

부록 0-6. 비용 관리

Vertex AI 는 사용량 기반 과금 모델을 사용합니다. 교육 실습에서 발생하는 비용을 이해하고 관리하는 방법을 안내합니다.

Vertex AI Gemini 모델 가격 체계

Gemini 모델은 입력/출력 토큰 수에 따라 과금됩니다 (2025 년 2 월 기준, 최신 가격은 GCP 공식 문서를 확인하세요):

모델	입력 (100 만 토큰당)	출력 (100 만 토큰당)	특징
Gemini 2.0 Flash	$0.10	$0.40	빠른 응답, 교육/개발에 최적
Gemini 2.0 Flash Lite	$0.025	$0.10	가장 저렴, 간단한 작업용
Gemini 1.5 Pro	$1.25 ~ $2.50	$5.00 ~ $10.00	고성능, 복잡한 추론
Gemini 2.5 Pro	$1.25 ~ $2.50	$10.00 ~ $15.00	최신 고성능 모델
text-embedding-005	$0.00002 (청크당)	-	텍스트 임베딩 생성

교육 실습 예상 비용

Gemini 2.0 Flash 기준으로 이 교육자료의 전체 실습을 수행할 때 예상되는 비용입니다:

실습 항목	예상 토큰 사용량	예상 비용
기본 SDK 호출 (Ch.2)	약 5 만 토큰	< $0.05
프롬프트 엔지니어링 (Ch.3)	약 10 만 토큰	< $0.10
RAG 실습 (Ch.4)	약 20 만 토큰 + 임베딩	< $0.20
Cloud Functions (Ch.5)	약 5 만 토큰	< $0.05
DB 연동 (Ch.6)	약 15 만 토큰 + 임베딩	< $0.15
전체 합계 (예상)	약 55 만 토큰	< $1.00

> **TIP:** 전체 교육 실습 비용은 $1 미만으로 예상됩니다. 무료 체험 크레딧($300)의 극히 일부만 사용하므로 비용 걱정 없이 학습할 수 있습니다.

기타 서비스 비용

서비스	과금 기준	무료 등급
Cloud Storage	저장 용량 + 네트워크	5GB 무료
Cloud Functions	호출 횟수 + 실행 시간	월 200 만 호출 무료
Cloud SQL (PostgreSQL)	인스턴스 시간 + 스토리지	없음 (Docker 로컬 사용 권장)
BigQuery	쿼리 처리 데이터량 + 스토리지	월 1TB 쿼리, 10GB 스토리지 무료

비용 절감 팁

Gemini 2.0 Flash 사용: 학습/개발 단계에서는 가장 비용 효율적인 Flash 모델을 사용하세요. Pro 모델은 복잡한 추론이 필요한 프로덕션에서만 사용합니다.

max_output_tokens 제한: 실습 코드에서 max_output_tokens 를 적절히 제한하면 불필요한 출력 비용을 줄일 수 있습니다.

Cloud SQL 대신 Docker: Chapter 6 의 pgvector 실습은 Docker 로컬 환경으로 충분합니다. Cloud SQL 은 프로덕션에서만 사용하세요.

사용 후 정리: 실습 후 불필요한 Cloud SQL 인스턴스, Cloud Functions 등을 삭제하여 지속 과금을 방지하세요.

크레딧 사용량 확인

무료 체험 크레딧 잔액을 확인하는 방법:

```
Cloud Console → 결제(Billing) → 개요(Overview)
→ 크레딧 잔액(Credits remaining) 확인

또는 결제 → 보고서(Reports)에서 서비스별 상세 사용량 확인
```

부록 0-7. 트러블슈팅

Vertex AI 실습 중 자주 발생하는 오류와 해결 방법을 정리합니다.

오류 1: API not enabled

```
에러 메시지:
403 Vertex AI API has not been used in project XXX before
or it is disabled.
```

원인: Vertex AI API 가 프로젝트에서 활성화되지 않았습니다.

해결:

```
# 방법 1: gcloud CLI
gcloud services enable aiplatform.googleapis.com

# 방법 2: Cloud Console
# API 및 서비스 → 라이브러리 → "Vertex AI API" 검색 → 사용(Enable)
```

오류 2: Permission denied

```
에러 메시지:
403 Permission 'aiplatform.endpoints.predict' denied on resource
또는
google.api_core.exceptions.PermissionDenied: 403
```

원인: 현재 인증된 계정에 Vertex AI 사용 권한이 없습니다.

해결:

```
# 1. 현재 인증 확인
gcloud auth list

# 2. ADC 재설정
gcloud auth application-default login

# 3. 필요 시 IAM 역할 부여
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="user:your-email@gmail.com" \
  --role="roles/aiplatform.user"
```

오류 3: Quota exceeded

```
에러 메시지:
429 Resource exhausted: Quota exceeded for
aiplatform.googleapis.com/generate_content_requests_per_minute
```

원인: 분당/일일 API 호출 할당량을 초과했습니다. 무료 체험 계정은 할당량이 제한됩니다.

해결:

```
# 방법 1: 잠시 기다린 후 재시도 (보통 1분 대기)
import time
time.sleep(60)  # 60초 대기

# 방법 2: 할당량 증가 요청
# Cloud Console → IAM 및 관리자 → 할당량(Quotas)
# → Vertex AI API 검색 → 편집(Edit) → 한도 증가 요청

# 방법 3: 코드에 재시도 로직 추가 (exponential backoff)
from tenacity import retry, wait_exponential
@retry(wait=wait_exponential(min=1, max=60))
def call_gemini(prompt):
    return model.generate_content(prompt)
```

오류 4: 프로젝트/리전 설정 오류

```
에러 메시지:
ValueError: Please provide a valid project ID
또는
404 Location us-central1 is not found or is not supported
```

원인: 프로젝트 ID 가 설정되지 않았거나 잘못된 리전을 지정했습니다.

해결:

```
# 1. 프로젝트 ID 확인
gcloud config get-value project

# 2. Python 코드에서 명시적 설정
```

```
import vertexai
vertexai.init(
    project="your-project-id",  # 프로젝트 ID (이름이 아님!)
    location="us-central1"       # Vertex AI 지원 리전
)

# 3. Vertex AI 지원 리전 목록:
# us-central1 (아이오와) – 가장 많은 기능 지원, 권장
# asia-northeast3 (서울) – 일부 기능 제한
# europe-west1 (벨기에) – 유럽 리전
```

오류 5: Python 패키지 버전 충돌

```
에러 메시지:
ImportError: cannot import name 'ChatVertexAI'
또는
AttributeError: module 'vertexai' has no attribute 'init'
```

원인: 패키지 버전이 오래되었거나 호환되지 않는 버전이 설치되어 있습니다.

해결:

```
# 1. 가상환경 재생성 (가장 확실한 방법)
deactivate                  # 기존 가상환경 비활성화
rm -rf venv                 # 가상환경 삭제
python -m venv venv         # 새로 생성
source venv/bin/activate    # 활성화
pip install -r requirements.txt  # 패키지 재설치

# 2. 특정 패키지 업그레이드
pip install --upgrade google-cloud-aiplatform
pip install --upgrade langchain-google-vertexai
```

오류 6: Docker/pgvector 관련 (Chapter 6)

```
에러 메시지:
docker: Cannot connect to the Docker daemon
또는
psycopg2.OperationalError: could not connect to server
```

원인: Docker 가 실행되지 않았거나 PostgreSQL 컨테이너가 시작되지 않았습니다.

해결:

```
# 1. Docker 실행 확인
docker info

# 2. pgvector 컨테이너 상태 확인
docker-compose ps
```

```
# 3. 컨테이너 재시작
docker-compose down
docker-compose up -d

# 4. 컨테이너 로그 확인
docker-compose logs pgvector-db

# 5. DB 접속 테스트
docker exec -it pgvector-db psql -U vectoruser -d vectordb
```

일반 디버깅 체크리스트

체크 항목	확인 명령어
gcloud 인증 상태	gcloud auth list
현재 프로젝트	gcloud config get-value project
활성화된 API	gcloud services list --enabled | grep aiplatform
Python 버전	python --version (3.9+ 필요)
패키지 설치 확인	pip list | grep google-cloud-aiplatform
환경변수 확인	echo $GOOGLE_CLOUD_PROJECT
네트워크 연결	curl -s https://aiplatform.googleapis.com/ > /dev/null && echo OK
Docker 상태	docker info && docker-compose ps

TIP: 문제 해결이 어려울 때는 Google Cloud 공식 문서(https://cloud.google.com/vertex-ai/docs/general/troubleshooting)를 참고하거나, Cloud Console 의 Cloud Shell(웹 브라우저 내 터미널)에서 동일한 코드를 실행해보세요. Cloud Shell 은 인증과 SDK 가 사전 설정되어 있어 환경 문제를 배제하고 테스트할 수 있습니다.